Stage 2: Service-Oriented Search Engine (SOA)

Subject: Big Data
Degree: Grado en Ciencia e Ingeniería de Datos (GCID)
Institution: Escuela de Ingeniería Informática, Universidad de Las Palmas de Gran Canaria (ULPGC)
Academic Year: 2026/2027
Team: TheScratchers
Repository: https://github.com/TheScratchers/stage_2

Authors and Responsibilities

Student	Student ID	Contact	Module Responsibilities	Report Contributions
Pablo Martínez Suárez	TO FILL	pablomrtinezzz@gmail.com	Ingestion Service, Embedded ActiveMQ Broker, Execution Scripts, Ingestion Benchmarks	Cover, Introduction, Architecture, Ingestion Implementation, Ingestion Benchmarks, Final PDF Compilation
Amado Artiles Rodríguez	TO FILL	amadoart22@gmail.com	Indexing Service, Broker Consumers, Concurrency & Idempotent Transactions, JMH Benchmarks	Indexing Implementation, Indexing Concurrency, JMH Microbenchmarks
Aimar Alejandro Santana	TO FILL	aimarsaal@gmail.com	Search Service, OpenAPI 3.0 Specification, API Validation, Load Client and Integration Benchmarks	API & Contracts, Search Implementation, Design Decisions, Integration Benchmarks, Conclusions & Stage 3

Reference Execution Environment

All official benchmark figures and system measurements reported in this document were executed under strictly controlled conditions on the agreed team reference machine:

- **Processor (CPU):** Intel Core i7-13700KF (16 physical cores: 8 Performance-cores @ 3.40–5.40 GHz + 8 Efficient-cores @ 2.50–4.20 GHz, 24 logical processors).
- **RAM:** 32 GB (2 × 16 GB) Kingston FURY Beast DDR5 5200 MHz CL40.
- **Motherboard:** MSI B760 GAMING PLUS WIFI.
- **Storage:** 1 TB NVMe M.2 SSD (Kioxia Exceria G2, PCIe 3.0 x4).
- **Operating System:** Windows 11 Pro 64-bit (build 10.0, amd64).
- **Java Runtime Environment (JVM):** Eclipse Adoptium Temurin 21.0.12.1+1 LTS (OpenJDK 64-Bit Server VM).
- **Build System:** Apache Maven 3.9.16 (language level `<release>17</release>`).
- **Baseline Commit:** `04132ab` (official measurement freeze).

Table of Contents

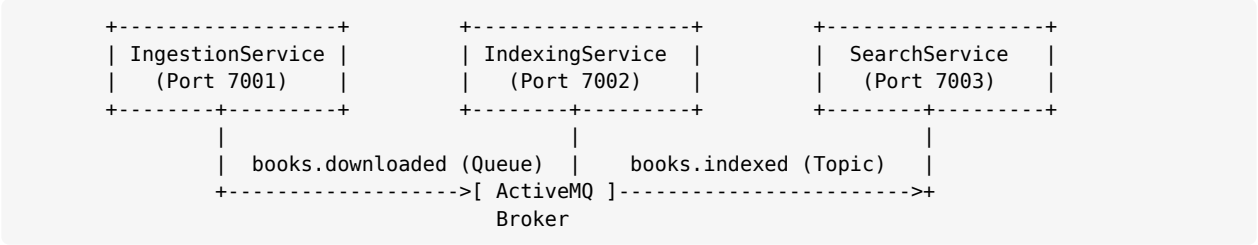
2. Introduction and Objectives 3
2.1 Architectural Context 3
2.2 System Objectives and Non-Functional Goals 3
3. System Architecture and Design 5
3.1 Global System Overview 5
3.2 Event Choreography vs. Central Orchestration 7
3.3 Storage Isolation and the Single-Writer Principle 7
3.4 Concurrency and Threading Architecture 8
4. API design and contracts 10
4.1 Front-end use cases and endpoints 10
4.2 Internal and coordination interfaces 10
4.3 Common conventions 11
4.4 Validation evidence 12
5. Microservice implementation 13
5.1 Ingestion Service 13
5.2 Indexing Service 16
5.3 Search Service 19
6. Design decisions 22
6.1 Coordination mechanism: choreography through a message broker 22
6.2 Single-machine deployment 22
6.3 API design 23
6.4 Storage 23
6.5 Concurrency model, replicas, threads and ports 24
6.6 Where replication and threading help, and where they cause contention 24
7. Benchmarks and results 26
7.1 Ingestion Throughput Benchmarks 26
7.2 Microbenchmarks of the Indexing components (JMH) 28
7.3 Integration benchmarks 31
8. Conclusions and future improvements 36
Assumptions that depend on a single host 36
How the architecture will evolve 36
What the Stage 2 measurements tell us 37

2. Introduction and Objectives

2.1 Architectural Context

In Stage 1 of this project, we designed and implemented a standalone data layer for a Gutenberg-based search engine. That initial system processed literary works sequentially through batch execution, demonstrating core text extraction, inverted indexing, and keyword search. However, a monolithic design inherently couples all stages into a single lifecycle: an error during text downloading aborts downstream indexing, concurrent read and write operations compete directly within the same runtime heap, and scaling individual components independently is impossible.

Stage 2 transforms that foundation into a robust **Service-Oriented Architecture (SOA)** composed of autonomous, cooperating microservices implemented in Java 17. The entire system is engineered to operate on a single host machine while adhering to strict distributed systems principles, establishing the exact communication patterns, data ownership boundaries, and fault-tolerance semantics required for seamless transition to a multi-node cluster in Stage 3.



The system is organized into three decoupled services:

1. **Ingestion Service:** Responsible for external data acquisition. It interfaces with Project Gutenberg (or a local mirror), fetches full literary texts, strips raw metadata from textual bodies, stores raw artifacts atomically into the datalake, and announces available books through the message broker.
2. **Indexing Service:** Responsible for text analysis and datamart generation. It consumes ingestion events, parses structural Gutenberg headers into standardized metadata (title, author, ISO language, publication year), tokenizes document bodies into lexical terms, and constructs an inverted index persisted into a local SQLite datamart.
3. **Search Service:** Responsible for serving user-facing search queries. It exposes a public RESTful API providing keyword search with multi-field filtering (author, language, year), relevance ranking, deterministic pagination, and hit details retrieval. It operates strictly read-only against the datamart and dynamically updates in-memory filter caches upon receiving indexing events.

An independent **ActiveMQ Message Broker** process provides asynchronous choreography, serving as the central nervous system that coordinates work between services without direct inter-service HTTP invocations.

2.2 System Objectives and Non-Functional Goals

The architectural design of Stage 2 was governed by a concrete set of functional and non-functional requirements:

1. Loose Coupling and Choreography

Services must operate without compile-time or runtime coupling to each other's network addresses or internal storage representations. Ingestion must not call Indexing over HTTP, nor should Indexing instruct Search to refresh. Instead, services communicate through strongly-typed, versioned event schemas transmitted over persistent message queues and publish-subscribe topics.

2. Strict Data Ownership (Single-Writer Principle)

To prevent concurrency hazards and uncoordinated file mutations, every persistent file on disk is assigned exactly one owning service:

- The **Datalake** (`DATA_DIR/datalake/`) and the state database `ingestion.db` are written exclusively by the Ingestion Service.
- The **Datamart** (`DATA_DIR/datamarts/datamart.db`) is written exclusively by the Indexing Service.
- The **Search Service** maintains zero persistent disk state and opens the datamart through read-only SQLite connections (`common.Sqlite.openReadOnly`), so it can never modify it.

3. End-to-End Idempotency and Fault Tolerance

In distributed environments, network blips, process crashes, and message duplicates are inevitable. The pipeline guarantees full idempotency across every stage:

- Repeated ingestion requests for the same book detect existing records in SQLite and bypass redundant downloads.
- Re-indexing an existing book executes inside an atomic SQLite transaction that deletes previous postings before inserting updated frequencies, ensuring that duplicate broker deliveries leave the index in a consistent state.
- Unacknowledged messages resulting from service crashes during indexing are automatically redelivered by ActiveMQ. After five consecutive failures, problematic messages are routed to `ActiveMQ.DLQ` (Dead Letter Queue) rather than blocking the pipeline.

4. High Concurrency and Predictable Latency

- The Search API must sustain concurrent user traffic with bounded response latencies (targeting low-millisecond latencies under moderate loads).
- The Ingestion pipeline must support parallel client requests without corrupting the datalake or starving system I/O.
- The Indexing Service must exploit multi-core architectures through concurrent worker pools while respecting SQLite's single-writer database lock.

5. Measurable Performance and Empirical Validation

Every architectural claim, concurrency speed-up, and resource footprint must be substantiated through repeatable, automated benchmarks executed against a fixed reference corpus (the 20 canonical books from Stage 1) on dedicated reference hardware.

3. System Architecture and Design

3.1 Global System Overview

The Stage 2 Search Engine is architected as an event-driven collection of autonomous microservices executing within independent Java Virtual Machines on a single host. Communication is strictly partitioned between two distinct tiers:

1. **Public Front-End Tier:** Exposed exclusively by the Search Service over HTTP REST (port 7003), providing read-only endpoints consumed by human readers, client applications, and front-end user interfaces.
2. **Internal Choreography Tier:** Coordinated asynchronously through an embedded Apache ActiveMQ message broker (TCP port 61616). Services never initiate direct inter-service HTTP calls; instead, business workflows progress through discrete, immutable events published onto persistent message queues and topics.

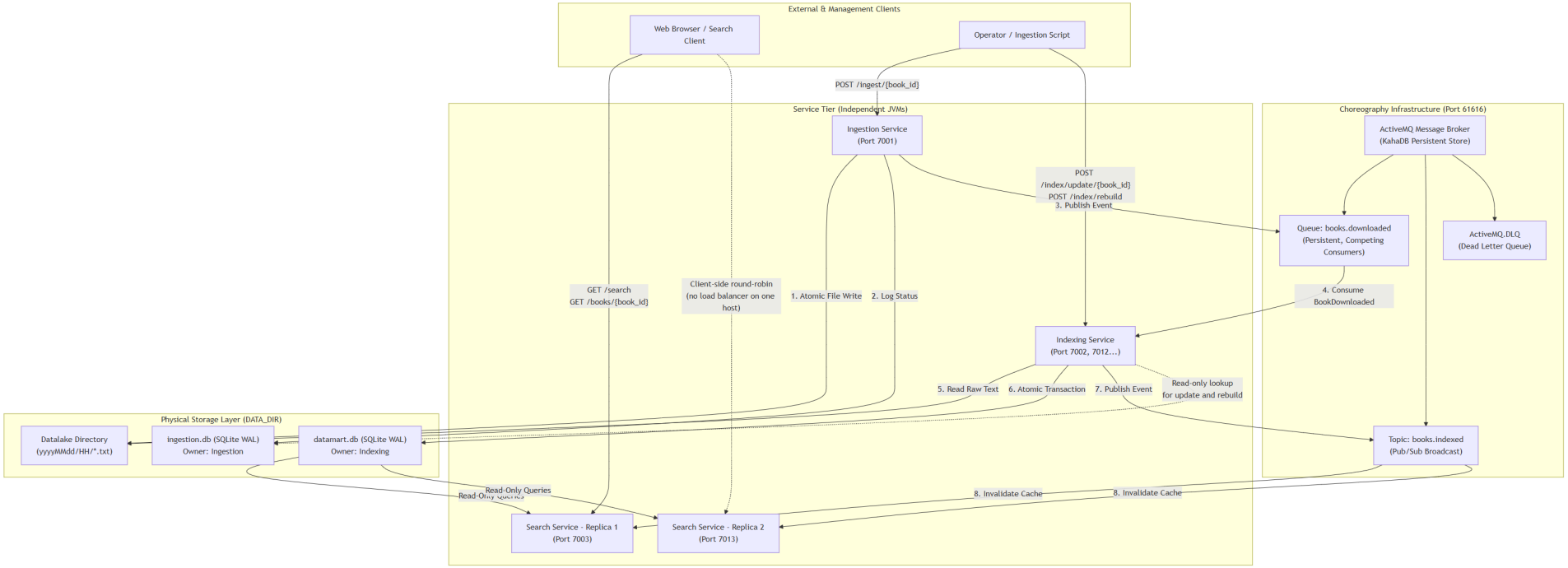


Figure 3.1. Processes, front-end and internal interactions, broker destinations and shared storage, all on one host (rendered from the Mermaid source of this section).

3.2 Event Choreography vs. Central Orchestration

A pivotal decision in Stage 2 was adopting an **event-driven choreography** rather than a centralized orchestrator (such as a master workflow coordinator issuing HTTP calls).

Architectural Quality	Central HTTP Orchestrator	Event Choreography (ActiveMQ)
Coupling	High: orchestrator must know network addresses and ports of all downstream services.	Low: services only know the broker URL (`tcp://localhost:61616`) and event schemas.
Resilience to Crashes	Fragile: if the orchestrator crashes mid-pipeline, state is lost unless backed by external consensus.	Robust: messages remain safely persisted in KahaDB until explicitly acknowledged after downstream commit.
Backpressure Handling	Difficult: requires rate-limiting algorithms, circuit breakers, and complex HTTP retry loops.	Natural: consumers pull messages at their own processing rate (`prefetch=1`); queues absorb sudden traffic bursts.
Horizontal Scaling	Requires service discovery and client-side load balancers for internal endpoints.	Trivial: spinning up multiple Indexing or Search JVMs automatically balances load via competing consumers.

Messaging Topology and Semantics

The message broker enforces two distinct messaging patterns tailored to workflow requirements:

1. **`books.downloaded` (Queue - Point-to-Point):**
   - **Delivery Mode:** `PERSISTENT` (backed by KahaDB write-ahead logs on disk).
   - **Distribution:** Competing Consumers pattern. Worker threads across one or more Indexing processes compete for messages.
   - **Prefetch Policy:** Set to `prefetch=1`. This prevents a single fast thread or worker from hoarding batches of books while other workers sit idle, ensuring optimal load distribution across multi-threaded workers.
   - **Acknowledgement Mode:** Transacted JMS session (`SESSION_TRANSACTED`), one per worker thread. The consumer commits the session, which acknowledges the message and releases its `BookIndexed` event, **only after** the SQLite transaction with the book's metadata and postings has committed. If indexing fails, the session is rolled back; if the process crashes, the broker detects the lost connection. In both cases the message is redelivered to an available worker.
   - **Dead Letter Strategy:** The broker's `RedeliveryPlugin` redelivers a failed message up to 5 times with a fixed delay of 2 s (no exponential backoff); client-side redelivery is disabled so that only the broker counts attempts. Poison messages (unreadable JSON, invalid events or missing files) then go to `ActiveMQ.DLQ` without halting the ingestion or indexing pipelines.
2. **`books.indexed` (Topic - Publish/Subscribe):**
   - **Delivery Mode:** Broadcast to all active subscribers.
   - **Distribution:** Fan-out to every running Search Service replica.
   - **Purpose:** Informs Search replicas that a new book has entered the datamart, prompting them to invalidate their in-memory filter caches (distinct languages and publication years). Searches themselves read committed SQLite WAL snapshots directly from disk, so they never depend on the events. The subscription is not durable: while a replica is disconnected from the broker, or before it has subscribed after a restart, its cache is disabled and every request reads the datamart. On reconnection the cache is emptied and enabled again, so missed events can never leave stale filter values.

3.3 Storage Isolation and the Single-Writer Principle

Data storage is partitioned into separate directories under `DATA_DIR`, one per owner:

```
DATA_DIR/
├── broker/                 (Owned by the ActiveMQ broker)
│   └── kahadb/             (KahaDB journal files and indexes: db-1.log, db.data, ...)
```

```
├── datalake/                (Owned exclusively by Ingestion Service)
│   ├── ingestion.db         (SQLite database: book_id, status, paths, timestamp)
│   ├── ingestion.db-wal
│   ├── ingestion.db-shm
│   └── yyyyMMdd/
│       └── HH/
│           ├── 1342.header.txt
│           └── 1342.body.txt
└── datamarts/               (Owned exclusively by Indexing Service)
    ├── datamart.db          (SQLite database: books, postings, index_meta)
    ├── datamart.db-wal
    └── datamart.db-shm
```

The Single-Writer Principle

Concurrent file modification across autonomous processes is a notorious source of silent data corruption. Stage 2 resolves this by enforcing that **every file and database has exactly one designated writer process**:

- **Ingestion Service:** Only writer for the datalake text files and `ingestion.db`.
- **Indexing Service:** Only writer for `datamart.db`. It opens `ingestion.db` strictly read-only (a read-only SQLite connection from `common.Sqlite.openReadOnly`), only to locate the files of available books for the administrative operations `POST /index/update/{book_id}` and `POST /index/rebuild`.
- **Search Service:** Strictly read-only consumer of `datamart.db`. It never writes to disk.
- **ActiveMQ Broker:** Only writer for the `broker/` directory.

SQLite WAL Mode Concurrency

Both `ingestion.db` and `datamart.db` are configured with Write-Ahead Logging (`PRAGMA journal_mode=WAL`), `PRAGMA synchronous=NORMAL`, and `PRAGMA busy_timeout=5000`. WAL mode provides essential concurrency guarantees:

- **Readers never block writers**, and **writers never block readers**.
- Multiple Search service replicas can query `datamart.db` concurrently without experiencing lock contention while Indexing commits new book postings in the background.
- Concurrent SQLite writers are serialised via internal file locks. As established in the official benchmarks (Section 7), scaling Indexing worker threads beyond 2 workers saturates the single-writer lock; thus, true parallelism occurs during in-memory text tokenization, while database commits queue safely.

3.4 Concurrency and Threading Architecture

Every service runs in its own JVM process with a dedicated threading model tailored to its operational workload:

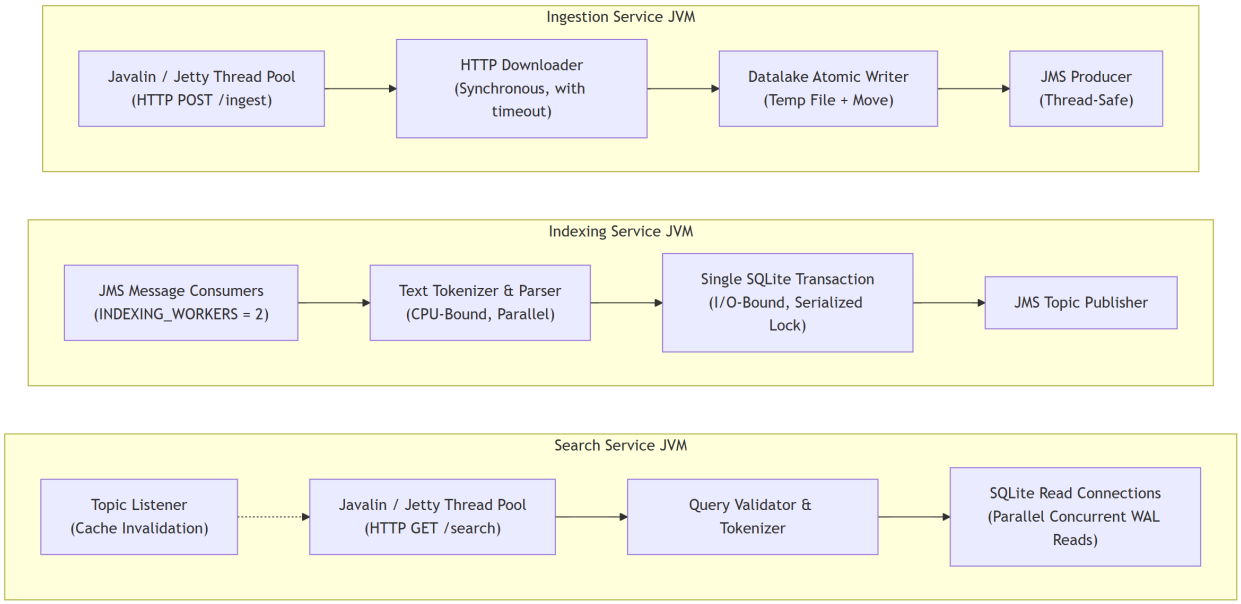


Figure 3.2. Threads and processing steps inside each service JVM.

1. **Ingestion Service:**
   - Jetty HTTP server manages incoming client ingestion requests across an adaptive thread pool.
   - Text downloading and file writing are stateless and execute concurrently across threads.
   - Each thread obtains its own connection to `ingestion.db`, relying on SQLite WAL mode to coordinate the brief record inserts.
2. **Indexing Service:**
   - Spawns `INDEXING_WORKERS` concurrent JMS consumer sessions (default: 2 threads).
   - Each worker retrieves one `BookDownloaded` event (`prefetch=1`), independently reads the text files from the datalake, and tokenizes terms in memory.
   - Each book is stored in one SQLite transaction (delete the old postings, insert the new ones, upsert the book and `index_meta`), which waits up to `busy_timeout` if another worker is holding the write lock.
3. **Search Service:**
   - Handles read traffic using Jetty's worker threads.
   - Because queries only issue `SELECT` statements, SQLite WAL mode serves concurrent readers simultaneously through shared memory (`.shm`). Below saturation a search takes about 1–2.5 ms at the median (section 7.3).

4. API design and contracts

The system exposes two kinds of interface. A **front-end API**, provided by the Search Service, is the only one a user-facing application calls. **Internal interfaces** connect the services with each other and with the operators who run them. Following the choreography design, services never call each other over HTTP: Ingestion and Indexing coordinate through broker events, and the only internal HTTP endpoints are administrative ones used by scripts, operators and benchmarks.

The complete, versioned specification is `docs/openapi.yaml` (OpenAPI 3.0, version 1.0.0). It contains every route, parameter, schema, status code and example. The non-HTTP contracts (broker events, data ownership, SQLite schema and tokenizer) are in `docs/contracts.md`.

4.1 Front-end use cases and endpoints

A reader looking for books needs three operations, and the front-end API offers exactly these:

Use case	Endpoint	Provider	Consumers
Search books by keywords, optionally filtered by author, language and year, page by page	`GET /search`	Search Service (port 7003)	Search front-end, API checks, search benchmarks
Show the details of a book from the results	`GET /books/{book_id}`	Search Service	Search front-end, API checks
Offer the available languages and years as filter choices	`GET /search/filters`	Search Service	Search front-end, API checks

All three are read-only, idempotent and safe to retry. A front-end served from another origin can call them directly: the Search Service enables CORS for any origin (no credentials), answers preflight requests, and only exposes `GET` operations.

`GET /search` semantics. All parameters are query parameters.

Parameter	Rule	Default
`q`	Required, at most 200 characters, at least one word after tokenization. Words are combined with **AND**; repeated words count once	—
`author`	Case-insensitive substring of the author (`twain` matches `Mark Twain`); books without an author never match	none
`language`	Two-letter ISO 639-1 code, case-insensitive, normalised to lower case	none
`year`	Four-digit Project Gutenberg release year (the e-book header has no original publication year)	none
`offset`	0 to 999 999 999	0
`limit`	1 to 100	20

Filters combine with each other and with the query, and empty filter values are ignored. Results are ordered by **score**, the sum of the frequencies of the query words in the book, from highest to lowest; ties are broken by ascending `book_id`. The order is therefore deterministic and pages never overlap. The response separates `total` (all matching books) from `results` (the requested page) and echoes the normalised `filters`. A valid query with no matches returns `200` with an empty list. An `offset` beyond `total` also returns an empty page and keeps `total`.

Example, recorded from the real system loaded with the fixed corpus:

```
GET /search?q=darcy&language=EN&year=1998
200 {"query":"darcy","terms":["darcy"],"filters":
{"language":"en","year":1998},"total":1,"offset":0,
     "limit":20,"results":[{"book_id":1342,"title":"Pride and Prejudice","author":"Jane Austen",
     "language":"en","year":1998,"score":432}]}
```

4.2 Internal and coordination interfaces

Broker events (non-HTTP). The main internal interactions are two events on the ActiveMQ broker. Both are JSON with `snake_case` fields and a `version`, and both are sent as persistent messages.

Destination	Type	Producer → consumers	Payload	Meaning
`books.downloaded`	queue	Ingestion → Indexing (competing consumers)	`event`, `version`, `book_id`, `header_path`, `body_path`, `ingested_at`	Both files of the book are complete in the datalake and recorded in the ingestion log
`books.indexed`	topic	Indexing → every Search replica	`event`, `version`, `book_id`, `terms`, `indexed_at`	The book has been committed to the datamart

The rules that make this choreography reliable are part of the contract:

- **Acknowledgement after commit.** Indexing acknowledges a `BookDownloaded` message only after the SQLite transaction of the book has committed. It does so in the same JMS transaction that publishes `BookIndexed`. If anything fails, the message returns to the broker.
- **Retries and dead letters.** The broker redelivers a failed message up to 5 times and then moves it to `ActiveMQ.DLQ`.
- **Duplicates.** Redelivered or repeated events are harmless, because indexing a book replaces its previous version.
- **Ordering.** No order between different books is assumed.

Administrative HTTP endpoints. These are not meant for the front-end and do not enable CORS.

Endpoint	Provider	Consumers	Completion	Idempotency and retries
`POST /ingest/{book_id}`	Ingestion (7001)	Corpus-loading scripts, operators	Synchronous: on `200` the files are in the datalake, the log says `available` and `BookDownloaded` has been published	Re-ingesting an available book downloads nothing and re-publishes the event (`already_available`); safe to retry
`GET /ingest/status/{book_id}`, `GET /ingest/list`	Ingestion	Scripts, operators	—	Read-only
`POST /index/update/{book_id}`	Indexing (7002)	Operators, recovery	Synchronous: on `200` the new version is committed and visible to Search	Replaces the book; safe to retry
`POST /index/rebuild`	Indexing	Operators, benchmarks	Synchronous; Search never sees an empty or partial index	Re-runnable; `409` if one is already running in that process
`GET /index/status`	Indexing	Scripts, benchmarks	—	Read-only
`GET /health`	Every service	Start-up scripts, tests, benchmarks, monitoring	Liveness of the process only	Read-only

Callers of `POST /ingest` must expect a slow upstream. The Gutenberg download has a configurable timeout (`GUTENBERG_TIMEOUT_SECONDS`, 30 s by default), so they should allow at least that long. The failure responses tell the caller what to do:

- `404 NOT_FOUND`: the book does not exist on Gutenberg. Retrying returns the same error.
- `502 UPSTREAM_ERROR`: Gutenberg failed or timed out. Retrying later may succeed.
- `503 UNAVAILABLE`: the broker could not take the event. The book stays stored, and retrying the same request publishes the event again.

4.3 Common conventions

- **Format.** JSON in UTF-8, `snake_case` fields, `book_id` as a positive integer and timestamps in ISO-8601 UTC. Missing metadata is returned as an explicit `null`.
- **Errors.** Every error, in every service, uses one envelope: `{"error": {"code":"INVALID_PARAMETER","message":"limit must be between 1 and 100"}}`.
- **Status codes.**

Status	Code	Used for
400	`INVALID_PARAMETER`	Invalid input

Status	Code	Used for
404	`NOT_FOUND`	Unknown book
409	`CONFLICT`	Rebuild already running
502	`UPSTREAM_ERROR`	Gutenberg failure
503	`UNAVAILABLE`	Datamart not created yet, or broker down
500	`INTERNAL_ERROR`	Unexpected errors; internal details are only logged

- **Health.** Every process answers `GET /health` with its name and `UP`. Start-up scripts and benchmarks poll it before sending work.

4.4 Validation evidence

The contract is checked by code that runs on every build, not only by inspection. Everything is in `api-tests/`, with instructions and expected results.

Evidence	What it shows	How it runs
`cases/search.json`, 44 checks over a hand-made six-book datamart	Keyword search, AND of words, every filter alone and combined, empty results, pagination, every invalid parameter with its exact message, book details, filter values, `null` metadata, CORS	Every `mvn verify`, against an in-process Search Service
`cases/corpus-search.json`, 20 checks over the real pipeline	The exact `book_id`s and scores returned for each word of `samples/words.txt`, cross-checked against an independent SQL query, and the specification's examples	`java -jar api-tests/target/api-tests.jar pipeline`
`cases/ingestion.json` (10) and `cases/indexing.json` (9)	Internal operations and their completion and error behaviour: idempotent re-ingestion and update, unknown and invalid ids, a missing book on Gutenberg, a full rebuild	Same command
The pipeline itself	The broker contract end to end: books reach Indexing and Search only through `BookDownloaded` and `BookIndexed`	Same command
`OpenApiContractTest`	No checked route or status code is missing from `openapi.yaml`, every documented operation is exercised, and every `GET /search` example equals the real response	Every `mvn verify`
Broker and failure tests	Ingestion answers `503` and keeps the book when the broker is down; Indexing acknowledges only after commit, a killed Indexing process leaves no half-indexed book, and 1, 2 and 4 workers or 2 replicas build an identical index; Search keeps answering and stops caching while the broker is down	Every `mvn verify` (service test suites)

The pipeline command starts the four services from their jars and loads the 20 corpus books. It ends with `39/39 checks passed`. The books are served offline from `samples/books` by a local stand-in for Project Gutenberg, so the expected results do not depend on the network. When the same checks were run against the live site, every `book_id` and score still matched, but one title differed: Gutenberg had edited book 84 after the corpus was captured in Stage 1. This is why every experiment of this report uses the fixed corpus.

5. Microservice implementation

5.1 Ingestion Service

The Ingestion Service is the entry point for external data into the search engine. It acquires literary works from Project Gutenberg, parses and separates raw texts into structured headers and bodies, stores artifacts atomically in the datalake, records status in a local SQLite ledger, and emits choreography events to notify downstream services.

It is implemented as a lightweight Java 17 service utilizing Javalin 6.1.3 for its HTTP server, Google Gson 2.11.0 for JSON serialization, `sqlite-jdbc` 3.47.1.0 for local state management, and Apache ActiveMQ 6.3.2 client libraries for event publishing. The service is packaged as an autonomous executable jar (`ingestion-service/target/ingestion-service.jar`).

It is configured through environment variables or system properties via `common.Config`:

Configuration Variable	Default Value	Description
`INGESTION_PORT`	`7001`	HTTP listening port
`DATA_DIR`	`./data`	Root path for datalake directories and SQLite databases
`BROKER_URL`	`tcp://localhost:61616`	ActiveMQ broker connection URL
`GUTENBERG_BASE_URL`	`https://www.gutenberg.org`	Base URL for fetching Gutenberg texts (can be pointed to a local mirror)
`GUTENBERG_TIMEOUT_SECONDS`	`30`	HTTP client read and connect timeout
`DATALAKE_LAYOUT`	`time_based`	Datalake folder organization strategy (`time_based`: `yyyyMMdd/HH/`)

5.1.1 Internal Structure and Package Organization

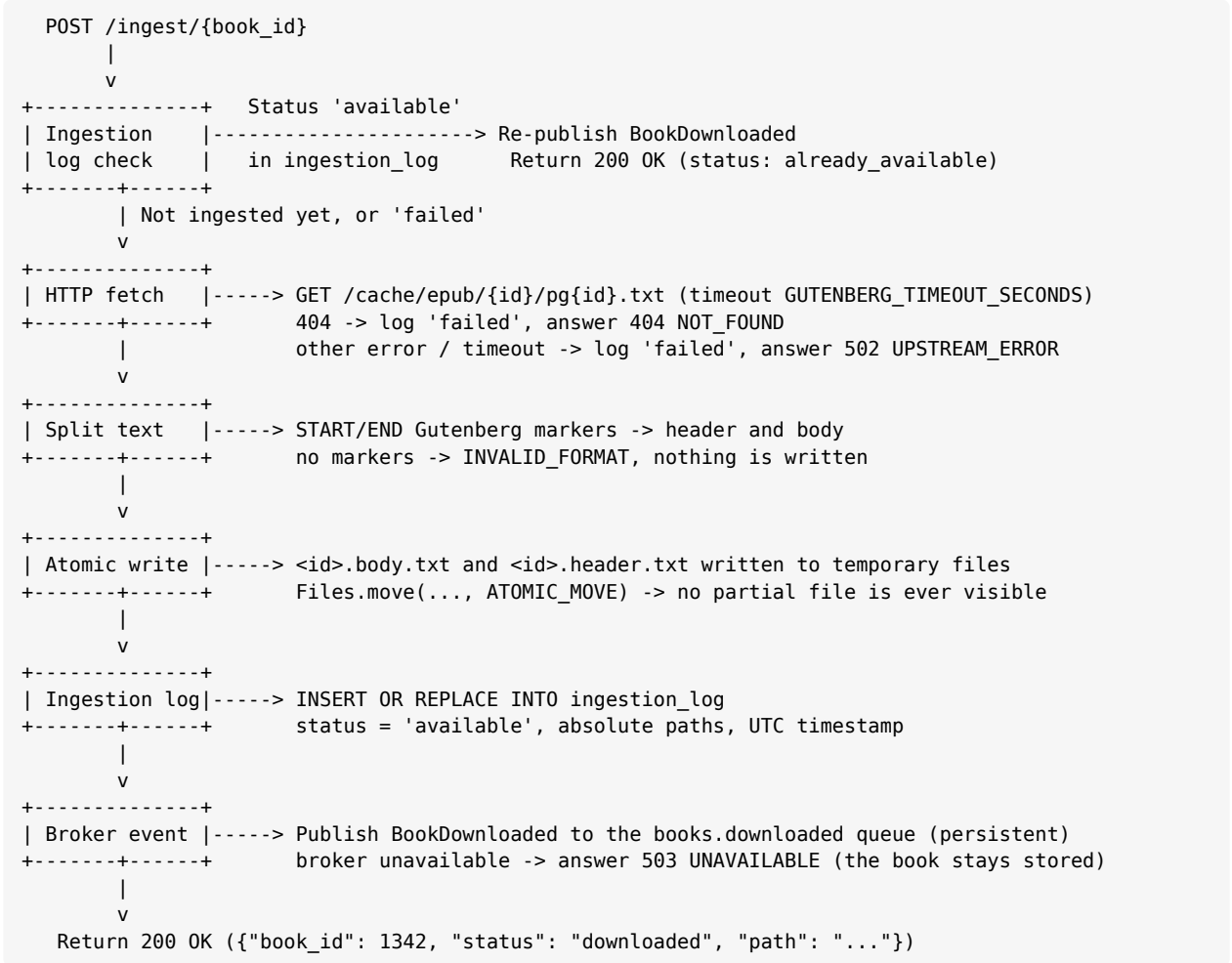
The codebase is organized into decoupled packages with single responsibilities and downward-only dependencies:

Package	Class / Component	Responsibility
`ingestion`	`IngestionApp`	Service entry point; wires the dependencies from the configuration and maps the Javalin routes `/health`, `POST /ingest/{book_id}`, `GET /ingest/status/{book_id}` and `GET /ingest/list`, translating failures into the common error envelope.
`ingestion`	`IngestionService`	Core business logic: checks the ingestion log, delegates the download, records the result and publishes the `BookDownloaded` event.
`ingestion`	`BookDownloader`	Downloads one book, splits it and stores both files in the datalake (download → split → store). It keeps no state, so several threads can download different books at the same time.
`ingestion`	`IngestResult`	Immutable record returned by `POST /ingest/{book_id}` (`bookId`, `status`, `path`).
`ingestion.gutenberg`	`GutenbergClient`	HTTP client of Project Gutenberg (`/cache/epub/{id}/pg{id}.txt`). One shared `HttpClient`, with a connection and request timeout (`GUTENBERG_TIMEOUT_SECONDS`). It maps HTTP 404 to `NOT_FOUND` and any other status, timeout or network failure to `UPSTREAM_ERROR`.
`ingestion.gutenberg`	`GutenbergText`	Pure text processor: finds the START and END markers (case-insensitive, `THE` or `THIS`) and splits the text into header and body.

Package	Class / Component	Responsibility
`ingestion.gutenberg`	`SplitBook`, `DownloadException`	Header and body of a book; download failure with its reason (`NOT_FOUND`, `UPSTREAM_ERROR`, `INVALID_FORMAT`).
`ingestion.datalake`	`DatalakeLayout`	Computes the directory of a book from the ingestion time (UTC) and the configured layout (`yyyyMMdd/HH/` by default).
`ingestion.datalake`	`DatalakeWriter`	Writes each file atomically: temporary file, then atomic move to its final name.
`ingestion.datalake`	`IngestionLogStore`, `IngestionRecord`	SQL access to `ingestion_log` in `DATA_DIR/datalake/ingestion.db` (`book_id`, `status`, `header_path`, `body_path`, `ingested_at`).
`ingestion.broker`	`BookDownloadedPublisher`, `BookDownloadedEvent`	Publishes the persistent `BookDownloaded` JSON event on the `books.downloaded` queue, as specified in `docs/contracts.md`.

5.1.2 The Ingestion Pipeline: Step-by-Step Execution

A client or script ingests a book with `POST /ingest/{book_id}` (for example `POST /ingest/1342`; there is no request body). The service executes the following sequence:



1. Idempotency Fast-Path

Before any download, `IngestionService` looks the book up in `ingestion.db`. If it is already recorded as `available`, nothing is downloaded or written again: the service **re-publishes** the `BookDownloaded` event with the recorded paths and returns `200 OK` with `status: already_available`. Re-publishing makes a repeated request the recovery

procedure for an event that was lost; it is harmless because indexing a book is idempotent. As the official benchmarks show (section 7.1), this path answers in about 7–16 ms at the median, depending on the number of concurrent clients.

2. Network Acquisition

If the book is not available yet, `BookDownloader` fetches the raw text through `GutenbergClient`:

- The connection and the request have a timeout (`GUTENBERG_TIMEOUT_SECONDS`, 30 s by default), so a slow or silent server cannot block a request forever.
- There are no automatic retries inside the service. If Gutenberg answers with an error other than 404, times out or cannot be reached, the attempt is recorded as `failed` in `ingestion_log` and the client receives `502 UPSTREAM_ERROR`. It can simply repeat the request later.
- If the book does not exist (HTTP 404), the attempt is recorded as `failed` and the client receives `404 NOT_FOUND`; repeating the request gives the same answer.

3. Text Demarcation

Project Gutenberg texts surround the literary work with a header and a license. `GutenbergText` isolates it with the standard markers, matched case-insensitively and accepting both `THE` and `THIS`:

- **Header:** everything before the line `*** START OF THE PROJECT GUTENBERG EBOOK ... ***`.
- **Body:** everything between that line and `*** END OF THE PROJECT GUTENBERG EBOOK ... ***`.
- If a marker is missing, the text is rejected as `INVALID_FORMAT` and nothing is written to the datalake, so an incomplete or unexpected file never reaches Indexing.

4. Atomic Datalake Persistence

Writing a file directly to its final path could expose a truncated file if the process crashed mid-write. `DatalakeWriter` prevents it:

1. The book's directory (`DATA_DIR/datalake/yyyyMMdd/HH/` by default) is created if needed.
2. Each file is written to a temporary file in the same directory (`<id>.body.txt.<random>.tmp`, then the header).
3. `Files.move(..., ATOMIC_MOVE, REPLACE_EXISTING)` gives the temporary file its final name in one step. Readers see either the previous file or the complete new one, never a partial write. Storing a book again replaces its files.

5. Ingestion Log

The absolute paths of both files and the ingestion time are recorded in `DATA_DIR/datalake/ingestion.db` with one statement:

```
INSERT OR REPLACE INTO ingestion_log(book_id, status, header_path, body_path, ingested_at)
VALUES (?, 'available', ?, ?, ?);
```

The paths are absolute because other processes (Indexing) use them exactly as published, whatever their working directory.

6. Event Dispatch

After the log is updated, `BookDownloadedPublisher` serializes the `BookDownloaded` event and sends it to the ActiveMQ queue `books.downloaded` as a persistent message. The connection to the broker is opened lazily and re-created after a failure. If the broker cannot take the event, the client receives `503 UNAVAILABLE`. The book stays stored and recorded as `available`, so repeating the request publishes the event again through the fast-path above, and no book is lost.

5.1.3 Concurrency and Error Semantics

- **Thread Safety:** Javalin's Jetty threads serve the requests concurrently. `IngestionService` and `BookDownloader` hold no mutable state, every call opens its own SQLite connection, and the publisher synchronizes access to

its shared JMS connection, so several books can be ingested at the same time. Section 7.1 measures the effect of 1, 2 and 4 concurrent clients.

- **SQLite Concurrency:** `ingestion.db` uses the project's settings (`journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`). Status queries do not block, and the short write statements are serialized by SQLite, waiting up to 5 s for the lock.
- **Error Envelope:** Every HTTP error uses the project's common envelope, for example:

```
{"error": {"code": "NOT_FOUND", "message": "Book 99999999 does not exist on Project Gutenberg"}}
```

The codes used by the Ingestion Service are `400 INVALID_PARAMETER` (invalid id), `404 NOT_FOUND` (book missing on Gutenberg, or never ingested for the status endpoint), `502 UPSTREAM_ERROR`, `503 UNAVAILABLE` (broker down) and `500 INTERNAL_ERROR`, whose details are only logged.

5.2 Indexing Service

The Indexing Service turns the books of the datalake into the inverted index that Search reads. It is a Java 17 Javalin application with Gson for JSON, `sqlite-jdbc` for the datamart and the ActiveMQ client for the broker, packaged as an executable jar (`indexing-service/target/indexing-service.jar`). It is configured with `INDEXING_PORT` (default 7002), `DATA_DIR`, `BROKER_URL` and `INDEXING_WORKERS` (consumer threads, default 2), all read through `common.Config`.

It is the **only writer** of `DATA_DIR/datamarts/datamart.db`, which it creates on start-up. It reads the datalake files and, read-only, the ingestion log `DATA_DIR/datalake/ingestion.db`, both owned by Ingestion. It starts even if the broker is down and connects when it comes back.

Internal structure

Package	Class	Responsibility
`indexing`	`IndexingApp`	Wiring and start-up from the configuration; `/health`; closes the broker clients and the server on shutdown
`indexing`	`BookIndexer`	Indexes one book: parses the header, counts the terms of the body and stores both in one transaction
`indexing.metadata`	`HeaderParser`, `LanguageCodes`, `ParsedHeader`	Pure parsing of the Gutenberg header into title, author, ISO language code and year
`indexing.store`	`DatamartStore`	All SQL on `datamart.db`: schema, the per-book transaction, removal, counts and size
`indexing.source`	`IngestionLogReader`, `BookFiles`	Read-only lookup of available books and their file paths in `ingestion_log`
`indexing.service`	`IndexingService`	Logic of the administrative operations: update one book, rebuild, status
`indexing.api`	`IndexingController`, `ApiResponses`	HTTP layer: parameters, JSON in `snake_case` and the common error envelope
`indexing.events`	`BookDownloadedConsumer`, `BookIndexedPublisher`, event records	Consumption of `books.downloaded` and publication on `books.indexed`

The layers depend only downwards: the controller knows nothing about SQL, and `DatamartStore` knows nothing about HTTP or JMS. `BookIndexer` keeps no mutable state and `DatamartStore` holds no connection (every call opens its own), so one instance of each is shared by all the threads of the process.

How a book is indexed

1. **Locating the files.** The paths come from the `BookDownloaded` event, or, for the HTTP endpoints, from the `ingestion_log` row of an `available` book. Indexing never builds datalake paths itself, so the datalake layout stays a private decision of Ingestion. A missing file is an error, never an empty book.
2. **Metadata.** `HeaderParser` reads the header:
   - Title and author are matched at the start of a line, case-insensitively, including the indented continuation lines that Gutenberg uses for long values.

- The language is the first one listed on the `Language:` line, converted to an ISO 639-1 code (`English` ⟶ `en`), or `NULL` if it is missing or unknown.
- The year is the first four-digit number of the `Release date:` line before any `[`, so the eBook number is never taken for a year. It is the year of the Gutenberg release, not of the original work.

3. **Terms.** The body is split by `common.TextTokenizer`, which Search also uses for queries: lower case with `Locale.ROOT`, then maximal runs of Unicode letters, with no stemming or stop words. Each term is counted, and its **frequency** in the book is stored because Search ranks by the sum of the frequencies of the query terms.

4. **One transaction per book.** `DatamartStore.indexBook` runs, in one SQLite transaction:

```
DELETE FROM postings WHERE book_id = ?;                 -- re-indexing never duplicates
INSERT INTO postings(term, book_id, frequency) VALUES ... -- one batch for all the terms
INSERT OR REPLACE INTO books(...) VALUES (...);
INSERT OR REPLACE INTO index_meta VALUES ('last_update', ?);
COMMIT;
```

   Either all of the book is stored or nothing is. Indexing the same book twice leaves the same rows, which is what makes duplicate and redelivered events harmless.

5. **Publication.** After the commit, `BookIndexed` is published with the number of distinct terms.

Storage: the datamart

`datamart.db` holds three tables, with the schema of the team contract: `books` (metadata and `indexed_at`), `postings` (`term`, `book_id`, `frequency`, primary key `(term, book_id)`) and `index_meta` (`last_update`). The primary key serves the lookups of Search by term, and the extra index `idx_postings_book` serves the per-book `DELETE` of re-indexing.

SQLite settings (correction 7). Every connection uses `journal_mode=WAL`, `synchronous=NORMAL` and `busy_timeout=5000`, with the default page cache.

- **WAL** lets Search read while Indexing writes. Each reader keeps a snapshot, so it sees a book either before or after its commit, never half-indexed.
- **`synchronous=NORMAL`** does not sync the WAL on every commit. A committed book survives a crash of the process, but the last commits can be lost if the operating system crashes or the power fails. The corresponding messages may already be acknowledged by then, so they would not be redelivered. `POST /index/rebuild` restores the index from the ingestion log in that case. We accept this trade-off because the index can always be rebuilt from the datalake, which is the source of truth.
- **One writer at a time.** SQLite serialises the writers of a database: a writer waits up to `busy_timeout` for the lock. Worker threads and replicas therefore index in parallel but commit one after another. Section 7.2 measures what this costs.

Table 5.2.1. Size of the index for the 20-book corpus (1 MB = 10^6 bytes). Source: `benchmarks/results/20261105_indexing-jmh/footprint.csv` (1 thread).

Measure	Value
Corpus (header and body files)	11.29 MB
`books` rows	20
`postings` rows	139,869
`datamart.db` after a WAL checkpoint (tables and indexes)	7.19 MB

The whole index takes 64 % of the size of the text it indexes.

Coordination: consuming `BookDownloaded`

`BookDownloadedConsumer` starts `INDEXING_WORKERS` threads that compete for the `books.downloaded` queue. Each thread has its own **transacted** JMS session, because a JMS session must not be shared between threads. A message is handled in this order:

1. Parse and validate the event.

2. Index the book and commit it in SQLite.
3. Send `BookIndexed` on the `books.indexed` topic, inside the JMS transaction.
4. Commit the JMS session. This acknowledges the message and releases the event in one step.

If any step fails, the session is rolled back: nothing is published and the message returns to the broker. The broker redelivers it up to 5 times (6 deliveries in total) and then moves it to `ActiveMQ.DLQ`. Client-side redelivery is disabled, so that only the broker counts attempts.

What a crash leaves behind. A crash can happen at any point of that sequence:

- **Before the SQLite commit:** nothing of the book is stored, and the message is redelivered.
- **Between the SQLite commit and the JMS commit:** the book is stored, but `BookIndexed` has not been released and the message is not acknowledged. The message is redelivered, the book is re-indexed to the same rows, and the event is then published.
- **After the JMS commit:** the work is complete.

So every acknowledged message has a committed book, and `BookIndexed` is published at least once, always after the commit.

Other choices.

- **Prefetch 1.** A worker never holds messages it is not processing, so an idle worker or another replica can always take the next book. With the default prefetch, one busy consumer could hoard a batch while the others wait.
- **`failover:` transport.** The consumer connects in the background and reconnects after a broker restart, so the service never waits for the broker at start-up.
- **Invalid messages.** Unreadable JSON or an event without a valid `book_id` or paths fails like any other error and ends in the dead-letter queue, without stopping the worker.

Events after the HTTP endpoints. Books indexed by `POST /index/update` or `POST /index/rebuild` also publish `BookIndexed`, through `BookIndexedPublisher`, so that Search refreshes its filter cache in both cases. That publication runs inside an HTTP request after the commit, so it must neither fail nor block the request. It uses a direct connection with a 3 s timeout instead of `failover:`. If the broker is down, it logs a warning and tries again on the next publication. Search loses only the freshness of its cache until the next event (see the Search Service section).

Administrative operations

The contract of the endpoints is in section 4.2; this is how they behave:

- **`POST /index/update/{book_id}`** looks the book up in the ingestion log and indexes it as above. It answers `404` if Ingestion has not made the book available.
- **`POST /index/rebuild`** re-indexes every available book, one transaction per book, and then removes the indexed books that are no longer available. The final state is the same as indexing from scratch. Because the old version of each book stays until its new version commits, Search never sees an empty or partial index during a rebuild. A book that fails is reported in `failed_books` and keeps its previous version. A second rebuild in the same process answers `409`. Two replicas could rebuild at the same time, which wastes work but is harmless, because each book is still written in one transaction.
- **`GET /index/status`** reports the number of books, the time of the last committed change and the size of `datamart.db` with its WAL.

Concurrency: worker threads and replicas

There are two ways to index in parallel, and both compete for the same queue:

- **Threads.** `INDEXING_WORKERS` consumer threads per process. Each has its own JMS session and opens its own SQLite connections; JDBC connections are never shared between threads.
- **Replicas.** Several Indexing processes on different ports (7002, 7012, ...) over the same `DATA_DIR`. ActiveMQ delivers each message to only one consumer, whatever process it belongs to.

The only state shared between threads is immutable (the configuration and the stateless `BookIndexer` and `DatamartStore`), so there is nothing to lock in Java. The coordination happens in two places: the broker, which

hands each book to one worker, and SQLite, which serialises the commits. Reading, parsing and tokenising run fully in parallel; committing does not. No order between books is assumed, so the books may be committed in any order.

Failure and concurrency evidence

Automated tests check the guarantees above over the whole corpus (`IndexingRecoveryTest`). Every test sends the 20 books **twice**, so duplicates are always part of the workload, and compares the resulting datamart row by row with a reference index built sequentially. The comparison covers every `books` row except `indexed_at`, and every `postings` row.

Scenario	Check
A real `IndexingApp` process is killed abruptly (no shutdown hook) as soon as it has committed its first book	Messages were still pending; the datamart contains only complete books, with no half-indexed book and no orphan postings
A new process is started on the same queue and datamart	It completes the batch: the index equals the reference, nothing is duplicated and the dead-letter queue is empty
1, 2 and 4 worker threads in one process	The index equals the reference in every case
Two `IndexingApp` processes with 2 workers each, sharing one `datamart.db`	The index equals the reference, and both processes indexed books
A book whose files are missing (`BookDownloadedConsumerTest`)	It is delivered 6 times, ends in `ActiveMQ.DLQ` and nothing is indexed or published

The tests use a real ActiveMQ broker with the same redelivery policy as the project's broker. The killed and the replicated services are the real `IndexingApp`, each in its own JVM. Throughput with threads and replicas on the running system is measured in section 7.3.

Tests

The service has 62 tests:

Test class	Tests	What it covers
`HeaderParserTest`	9	Titles and authors on several lines, ISO language codes, release year, missing fields
`LanguageCodesTest`	3	Language names to ISO 639-1 codes
`DatamartStoreTest`	13	Schema in a missing directory, WAL, idempotent re-indexing, a failed transaction leaves no trace or keeps the previous version, removal, counts and size
`BookIndexerTest`	4	Indexing sample books from their files, frequencies, missing files
`IngestionLogReaderTest`	4	Available books only, missing ingestion log
`IndexingServiceTest`	8	Update, rebuild (removals, idempotency, books with missing files, a second rebuild while one runs), status, and events only for committed books
`IndexingControllerTest`	6	Every endpoint over HTTP, including `400` for an invalid id and `404` for a book that is not available
`BookDownloadedConsumerTest`	8	Real broker: indexing and publication, duplicates, dead-letter queue, invalid messages, 4 workers over the corpus, start without a broker
`BookIndexedPublisherTest`	2	Publication, and a broker that is down
`IndexingRecoveryTest`	5	The failure and concurrency scenarios of the previous section

On top of these, the API checks of section 4.4 exercise the running service.

5.3 Search Service

The Search Service implements the front-end API of section 4.1. It is a Java 17 Javalin application with Gson for JSON and `sqlite-jdbc` to read the datamart, packaged as an executable jar (`search-service/target/search-service.jar`). It is configured with `SEARCH_PORT` (default 7003), `DATA_DIR` and `BROKER_URL`. It never writes any data: it only reads the datamart that the Indexing Service owns, and it listens to `BookIndexed` events.

Internal structure

Package	Class	Responsibility
`search`	`SearchApp`	Wiring and start-up from the configuration; CORS; `/health`; closes the subscription on shutdown

Package	Class	Responsibility
`search.api`	`SearchController`, `ApiResponses`	HTTP layer: reads parameters, calls the core, writes JSON in `snake_case` with explicit `null`s, and maps each exception to the common error envelope and its status code
`search.query`	`SearchQuery`, `InvalidParameterException`	Validation and normalisation of the search parameters; the only place where the rules of section 4.1 are enforced
`search.store`	`DatamartReader`, `IndexUnavailableException`	Read-only SQL over `datamart.db`: search, book details and filter values
`search.cache`	`FilterValuesCache`	In-memory filter values, trusted only while the event subscription is up
`search.events`	`BookIndexedSubscriber`, `BookIndexedEvent`	Subscription to the `books.indexed` topic
`search.model`	`SearchHit`, `SearchPage`, `BookDetails`, `FilterValues`	Immutable results passed between the layers

The layers depend only downwards. The controller knows nothing about SQL, and the reader knows nothing about HTTP, so each one is tested on its own: validation, SQL against a hand-built datamart, HTTP against a running server, and the subscription against a real broker.

How a search is answered

1. **Validation.** `SearchQuery.parse` checks every parameter and normalises it: it trims `q` and the filters, lower-cases the language and ignores blank filters. A broken rule raises `InvalidParameterException`, whose message is returned with `400 INVALID_PARAMETER`.
2. **Tokenization.** `q` is split by `common.TextTokenizer`, the class the Indexing Service also uses to build the index. The query words are therefore produced by exactly the same rule as the indexed terms: lower case with `Locale.ROOT`, then runs of Unicode letters. The terms are de-duplicated, keeping the order of appearance.
3. **One SQL query for the matches.** A subquery keeps the books whose postings contain all the terms and sums their frequencies:

   ```
   SELECT book_id, SUM(frequency) AS score FROM postings
   WHERE term IN (?, ?, ...) GROUP BY book_id HAVING COUNT(*) = <number of terms>
   ```

   The primary key `(term, book_id)` of `postings` serves the `IN` lookup. The filters are applied on `books`, the result is counted for `total` and the page is read with `ORDER BY score DESC, book_id ASC LIMIT ? OFFSET ?`.
4. **Unicode-aware author filter.** SQLite's `lower()` only folds ASCII letters, so `BRONTË` would not match `Brontë`. `DatamartReader` therefore registers a small SQL function, `casefold`, that calls Java's `toLowerCase(Locale.ROOT)`, and filters with `instr(casefold(author), ?) > 0`.
5. **Consistent reads.** Every request opens its own read-only connection and runs the count and the page in **one read transaction**. In WAL mode a reader keeps a snapshot, so `total` and `results` always describe the same version of the index, even if Indexing commits a book in between. Search sees each book either before or after its commit, never half-indexed. Connections are never shared between Jetty threads.

Readiness and failure behaviour

- **Index not created yet.** Search starts even if the datamart does not exist yet, for example before Indexing has run. While the file or its tables are missing, every data endpoint answers `503 UNAVAILABLE`, and `/health` still answers `200`. Search never creates the datamart, so it cannot interfere with its owner.
- **Broker down.** Search also starts without a broker. The subscription uses ActiveMQ's `failover:` transport, connects in a background thread and reconnects with a growing delay after a broker restart, so searches never wait for the broker.
- **Unexpected errors.** They are logged with their stack trace and returned as `500 INTERNAL_ERROR` with a fixed message, so internal details never reach the client.

Publication of completed index updates

Searches read the datamart directly, so a book is searchable as soon as Indexing commits it. The `BookIndexed` events only matter for the one cached value, the filter choices of `GET /search/filters`, which would otherwise scan the `books` table on every call.

- **While subscribed.** `FilterValuesCache` keeps the values in memory, and every `BookIndexed` event drops them.
- **While the connection is interrupted.** Events could be missed, so the cache is disabled and every call reads the datamart. When the connection is restored, the cache is emptied and enabled again.
- **Races.** A generation counter prevents a slow read that started before an event from storing outdated values after it.

The topic subscription is not durable, so every replica receives every event while it is connected. Events missed while it is disconnected do not matter, because of the rule above.

Replicas and threads

The service is stateless apart from that cache, so any number of replicas can run on different ports over the same datamart. Each replica subscribes to the topic and keeps its own cache. Within a process, Javalin's Jetty thread pool serves requests concurrently. There is no shared mutable state apart from the cache, which is synchronised, and each request has its own SQLite connection. Read transactions in WAL mode do not block each other or the Indexing writer, so concurrency is limited by CPU rather than by locks. The integration benchmarks (section 7) measure this with 1 to 64 clients and with 1 and 2 replicas.

Tests

The service has 78 tests:

Test class	Tests	What it covers
`SearchAppHealthTest`	1	`/health`
`SearchQueryTest`	39	Every rule and message
`DatamartReaderTest`	18	AND, ordering with ties, each filter, pagination, `null` metadata, empty and missing index
`SearchControllerTest`	10	Exact documented bodies, every status code, CORS
`FilterValuesCacheTest`	5	The cache
`BookIndexedSubscriberTest`	5	Real ActiveMQ broker, including a broker restart

On top of these, the API checks of section 4.4 exercise the running service.

6. Design decisions

This section explains why the system is built the way sections 3 to 5 describe, which alternatives were rejected, and what the measurements of section 7 say about each choice.

6.1 Coordination mechanism: choreography through a message broker

The services coordinate through two events on an embedded ActiveMQ broker. `BookDownloaded` goes on a queue from Ingestion to Indexing, and `BookIndexed` goes on a topic from Indexing to every Search replica. No service calls another over HTTP.

Alternative	Why it was rejected
HTTP orchestration (a Control Module calls `POST /index/update` after each ingestion)	A coordinator has to track every book, retry failed calls and survive its own crashes. With events, that bookkeeping is done by the broker: a message stays until it is acknowledged
Scheduled polling (Indexing scans the datalake or the ingestion log for new books)	It adds a delay of up to one period to every book and keeps reading even when nothing changed. It also needs readiness markers so that a half-written book is never picked
Shared memory	It only works inside one host and couples the services' internal state, which would block the Stage 3 cluster

The broker gives the guarantees the pipeline needs with little code of our own:

- **Persistent messages** survive a broker restart.
- **Acknowledgement after commit:** a book whose indexing fails or crashes is redelivered, up to 5 times, and then parked in `ActiveMQ.DLQ` instead of being lost.
- **Competing consumers** on the queue spread the books over threads and replicas.
- **The topic** lets every Search replica hear about new books without knowing who indexed them.

Duplicates are allowed rather than prevented, because indexing a book replaces it (idempotency). That is simpler and more robust than exactly-once delivery.

Evidence.

- Re-indexing a book through the broker costs a median of 3.8 ms more than doing it directly, about 5 % of the work (Table 7.3.3). A new book is searchable about 64 ms after it is requested.
- The Indexing tests kill the process in the middle of a batch: no book is lost or half-indexed, and a restarted process completes the batch (section 5.2).

Cost. The broker is one more process to run, and a single point of failure on one host. Section 8 explains how it is replicated on the cluster.

6.2 Single-machine deployment

Every process runs on one host, as Stage 2 requires: one broker, one Ingestion, one or more Indexing and Search replicas, and the API checks and benchmarks. Each service is its own JVM started from its own jar, with its own port and configuration. They share only the broker and the files under `DATA_DIR`.

Separate processes, rather than one JVM with several modules, make the service boundaries real:

- a service can be stopped, restarted or replicated without touching the others;
- one service's crash does not take the others down;
- CPU and memory can be measured per service (Table 7.3.4).

Keeping everything on one host also keeps the measurements controlled. Every official figure comes from the same machine, JDK and commit.

The price is that every process competes for the same 24 logical processors, memory and disk. Some limits measured here are limits of the host and not of the design, such as the search ceiling of section 7.3.

6.3 API design

- **One public API.** Only the Search Service is meant for the front-end. Its three operations are `GET` requests on resources a reader understands: a search, a book and the filter choices. They are idempotent and cacheable, and CORS lets a browser call them from another origin. Everything else is internal and kept out of the front-end contract (section 4).
- **One search endpoint with optional filters,** as the assignment asks, rather than one route per filter.
- **Search semantics.**
  - Words are combined with **AND**, because adding a word should narrow the results.
  - Results are ranked by the **sum of term frequencies**, which needs no global statistics and can be computed in one SQL query.
  - Ties are broken by `book_id`, so the order is deterministic and pages never overlap or skip a book.
- **Bounded pagination.** `offset` and `limit`, with `limit` at most 100, so no request can ask for an unbounded page. `total` is always reported, so a client knows how many pages there are.
- **Strict, explained validation.** Invalid input is rejected with `400` and a message that names the rule, rather than silently corrected. Empty filters are ignored, because forms send them.
- **Uniform errors.** All three services return the same envelope and the same small set of codes, so a client handles errors once.
- **Synchronous internal operations.** `POST /ingest`, `/index/update` and `/index/rebuild` return when their work is committed. Scripts and tests therefore know the result without polling a status endpoint, and repeating a call is always safe.
- **Contract first.** `docs/openapi.yaml` and `docs/contracts.md` were started before the endpoints. Every service then documented its routes in the same pull request that implemented them. A test fails the build if a checked behaviour is undocumented, or if a documented example differs from the real system (section 4.4).

6.4 Storage

Data	Store	Writer	Readers
Book text	Datalake: header and body files under `DATA_DIR/datalake/`	Ingestion	Indexing
Which books are ready	`ingestion.db` (SQLite)	Ingestion	Indexing (read-only)
Inverted index and metadata	`datamart.db` (SQLite)	Indexing	Search (read-only)
Events in flight	KahaDB, the broker's store	Broker	—

Rule: one writer per file. Every file has exactly one service that writes it. This removes any need to coordinate writers across services and makes ownership explicit in the contract.

The datalake as files. Books are written once and read whole, so plain files are the simplest store. Each file is written to a temporary name and then moved atomically, so a reader never sees a half-written book. The datalake is also the source of truth: the index can always be rebuilt from it with `POST /index/rebuild`.

SQLite for the datamart. SQLite was the Stage 1 store, and it gives what the index needs without running a database server:

- **Transactions.** Each book is indexed in one transaction, so it is visible either whole or not at all, and re-indexing it never duplicates rows.
- **Fast lookups.** The `(term, book_id)` primary key answers searches.
- **Concurrent reads.** In WAL mode, Search reads while Indexing writes, each reader on a consistent snapshot.

The settings are `synchronous=NORMAL`, which survives a process crash and is faster than `FULL`, and `busy_timeout=5000`, so a writer waits for the lock instead of failing (section 5.2). Section 7 shows the result: the index takes 64 % of the size of its text (Table 7.3.5), and a search below saturation takes about 1–2.5 ms at the median (Table 7.3.1).

Its cost: one writer at a time. SQLite allows one write transaction per database. This is the limit that caps indexing at about 18.5 books per second, whatever the number of threads or replicas (Table 7.3.2). A server database or a partitioned index would remove this limit. On one host the simplicity was worth more than the extra throughput; on the cluster the index will have to be partitioned (section 8).

6.5 Concurrency model, replicas, threads and ports

Threads.

- **Search.** Javalin's Jetty thread pool serves requests concurrently. Each request opens its own read-only SQLite connection and runs in one read transaction.
- **Indexing.** `INDEXING_WORKERS` consumer threads (2 by default) compete for the queue. Each has its own JMS session and its own SQLite connections. The default is 2 because the second thread gives most of the measured gain and a third gives almost none (Table 7.3.2).
- **Ingestion.** Requests are served by Jetty threads.

Replicas and ports.

Process	Port	Replicable	How work is shared
Ingestion	7001	Not tested	One process; ingestion is bound by Project Gutenberg
Indexing	7002, then 7012, 7022, ...	Yes	Competing consumers on one queue, writing one datamart
Search	7003, then 7013, 7023, ...	Yes	Stateless apart from a cache; clients spread requests over replicas
Broker	61616	No (one per host)	—

Ports follow one pattern so that scripts can start replicas without a registry, and every port can be changed by configuration.

Protection of shared state. There is almost no shared mutable state, and what exists is guarded by the component that already owns it rather than by locks of our own:

Shared resource	Protection
A book in the queue	The broker delivers each message to one consumer (prefetch 1) and keeps it until acknowledged
`datamart.db`	SQLite serialises writers with its file lock; readers use WAL snapshots
Datalake files	One writer (Ingestion); atomic moves; consumers only read books announced as available
JDBC connections and JMS sessions	Never shared: one per request or per thread
Search filter cache	Synchronised, with a generation counter so that an outdated read never overwrites a newer invalidation
"One rebuild at a time"	An atomic flag per Indexing process; a concurrent rebuild in another replica is harmless because each book is still one transaction

The Indexing tests show that the result does not depend on the concurrency. 1, 2 and 4 threads, or 2 replicas, build the same index row by row, even with every book delivered twice (section 5.2).

6.6 Where replication and threading help, and where they cause contention

Work	Parallelism helps?	Evidence
Analysing books (reading, parsing, tokenizing)	**Yes**, almost linearly	JMH: 3.54× with 4 threads (section 7.2)
Ingesting books	**Yes**: each book writes its own files	From about 44 books/s with 1 client (runs 2–3; the first run includes the JVM warm-up) to about 112 with 4 clients (`20261105_ingestion-throughput/summary.csv`)
Writing the index	**No**: one writer per database	JMH: persisting stops improving after 2 threads (section 7.2); pipeline: 12.3 ⟶ 18.0 ⟶ 18.5 books/s with 1, 2 and 4 threads and 18.4 with 2 replicas (Table 7.3.2)
Answering searches	**Up to the host's limit**	Peak about 5,500 requests/s at 16 clients; a second replica on the same host does not raise it (Table 7.3.1)

- **Where it helps.** Extra threads pay off where work is independent: analysing different books, ingesting different books, or serving different searches while CPU is free.
- **Where it causes contention.** It stops paying off where everything meets one resource. For Indexing that resource is the SQLite write lock: extra threads only wait for it (about 140 % CPU with 4 threads against 134 % with 2). For Search on one host it is the machine itself, shared with the load generator.

- **What this means for the configuration.**
  - Indexing runs **2 workers** by default: the best measured value per thread.
  - On one host, a second Search replica is worth running for **availability** (one can be restarted while the other answers) rather than for speed.
  - Ingestion's clients can safely send several books at once.

  On the cluster, threads remain a per-machine setting, and replicas on separate machines become the way to scale.

7. Benchmarks and results

7.1 Ingestion Throughput Benchmarks

The Ingestion Benchmark evaluates the throughput, latency distribution, scalability, and resource footprint of the Ingestion Service under concurrent load. It tests both fresh data ingestion (retrieval from a local Gutenberg stand-in, text demarcation, atomic datalake storage, SQLite ledger logging, and event dispatch) and idempotent re-ingestion (lookup in the ingestion log and re-publication of the event).

7.1.1 Method and Experimental Setup

All experiments were executed using the official benchmark harness (`IngestionBenchmark.java`) triggered via `scripts/run-ingestion-benchmark.ps1` on the reference machine.

Reference Environment:

Component	Specification
CPU	13th Gen Intel Core i7-13700KF (16 cores, 24 logical processors)
RAM	32 GB (2 × 16 GB) Kingston FURY Beast DDR5 5200 MHz CL40
Storage	1 TB NVMe M.2 SSD (Kioxia Exceria G2, PCIe 3.0 x4)
Operating System	Windows 11 Pro 64-bit (build 10.0, amd64)
JVM	Eclipse Adoptium Temurin 21.0.12.1+1 LTS (64-Bit Server VM)
Baseline Commit	`04132ab`
Execution Date	2026-11-05 (official benchmark milestone H6)

Controlled Experimental Conditions:

- **Fixed Corpus:** Ingestion operates against the canonical 20-book corpus defined in `samples/books.txt`.
- **Local Gutenberg Mirror:** Books are served directly from `samples/books/` by a local HTTP mock server. This guarantees that external internet latency or Gutenberg rate-limiting never pollutes the measurements.
- **Closed-Loop Concurrency:** The benchmark generator maintains a fixed number of concurrent client threads (1, 2, and 4 clients). Each thread submits a synchronous `POST /ingest` request, waits for the HTTP response, and immediately requests the next book without think time.
- **Fresh vs. Idempotent Passes:**
  - **Fresh Mode:** Starts with a clean datalake directory and an empty `ingestion.db`. All 20 books undergo complete downloading, text splitting, atomic file writes, SQLite insertion, and JMS message dispatch.
  - **Idempotent Mode:** Immediately re-ingests the same 20 books into the now-populated system. It exercises the fast path: the ingestion log is checked, nothing is downloaded or written, and the `BookDownloaded` event is published again.
- **In-process deployment:** Unlike the integration benchmarks of section 7.3, this harness runs the Ingestion Service (`IngestionApp.start`), the client threads and an embedded, **non-persistent** ActiveMQ broker (`vm://localhost`) inside one JVM. It therefore measures the service's own work without the cost of persisting events to KahaDB; that cost is part of the coordination measurements of section 7.3.
- **Repetitions:** Each concurrency level is measured in 3 consecutive runs. There is no separate warm-up phase, so the first run includes JVM class loading and JIT compilation; the warm-up effect is visible in run 1 with one client and is excluded from the means of Table 7.1.2.

7.1.2 Benchmark Results

Table 7.1.1. Ingestion throughput and request latencies for 1, 2, and 4 concurrent clients across 3 runs. Source: `benchmarks/results/20261105_ingestion-throughput/summary.csv`.

Clients	Run	Mode	Books	Errors	Duration (s)	Throughput (books/s)	Mean (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)
1	1	fresh	20	0	0.843	23.72	41.95	23.89	35.25	392.44	392.44
1	1	idempotent	20	0	0.194	102.89	9.66	8.18	10.07	32.95	32.95
1	2	fresh	20	0	0.454	44.03	22.66	20.35	37.06	39.09	39.09
1	2	idempotent	20	0	0.179	111.61	8.91	8.05	8.43	26.50	26.50
1	3	fresh	20	0	0.457	43.72	22.81	20.06	38.33	56.76	56.76
1	3	idempotent	20	0	0.165	121.55	8.18	7.05	8.50	25.41	25.41
2	1	fresh	20	0	0.267	74.93	25.50	24.38	37.93	39.12	39.12
2	1	idempotent	20	0	0.115	173.64	11.38	9.56	28.63	30.56	30.56
2	2	fresh	20	0	0.257	77.79	25.45	21.69	38.84	60.99	60.99
2	2	idempotent	20	0	0.120	166.21	11.87	9.80	30.01	32.30	32.30
2	3	fresh	20	0	0.286	69.95	27.84	25.26	40.01	44.67	44.67
2	3	idempotent	20	0	0.120	166.19	11.87	9.88	29.68	31.81	31.81
4	1	fresh	20	0	0.179	111.42	34.34	27.03	60.96	72.43	72.43
4	1	idempotent	20	0	0.115	173.22	22.09	14.71	37.49	39.90	39.90
4	2	fresh	20	0	0.176	113.88	33.75	28.33	61.34	63.47	63.47
4	2	idempotent	20	0	0.117	171.24	22.30	16.44	40.06	43.05	43.05
4	3	fresh	20	0	0.182	109.88	33.81	25.96	59.41	59.96	59.96
4	3	idempotent	20	0	0.115	174.37	22.05	14.38	40.08	42.16	42.16

Table 7.1.2. Aggregated throughput and speed-up, computed from Table 7.1.1. Fresh mode: mean of runs 2 and 3 at every level, because run 1 includes the warm-up of the JVM. Idempotent mode: mean of the 3 runs, which run on a warm JVM. Speed-up is relative to 1 client.

Mode	1 Client (books/s)	2 Clients (books/s)	4 Clients (books/s)	Speed-up 1 ⟶ 2	Speed-up 1 ⟶ 4
Fresh Ingestion	43.88	73.87	111.88	**1.68×**	**2.55×**
Idempotent Fast-Path	112.02	168.68	172.94	**1.51×**	**1.54×**

Table 7.1.3. Resource use of the benchmark JVM (Ingestion Service, client threads and embedded broker together) over the whole run, sampled every 250 ms. CPU is `OperatingSystemMXBean.getProcessCpuLoad()`, a share of the whole machine (100 % = all 24 logical processors). Memory is the Java heap in use (`totalMemory - freeMemory`), not the resident memory of the process. Source: `benchmarks/results/20261105_ingestion-throughput/resources-summary.csv` (columns `mean_cpu_percent`, `max_cpu_percent`, `max_rss_mb`).

Process	Mean CPU (% of the machine)	Peak CPU (% of the machine)	Peak Java heap in use (MB)
Benchmark JVM	7.9 %	28.2 %	279.1 MB

The resident memory of the deployed Ingestion process, measured from outside in the integration benchmarks, is about 293 MB (Table 7.3.4).

Table 7.1.4. Storage footprint on disk generated by ingesting the 20-book corpus. Source: `benchmarks/results/20261105_ingestion-throughput/disk.csv`.

Storage Store	Path / Description	Size on Disk
Datalake Text Artifacts	`DATA_DIR/datalake/yyyyMMdd/HH/*.txt` (40 files: 20 headers, 20 bodies)	11,290,405 bytes (11.29 MB)
Ingestion SQLite Ledger	`DATA_DIR/datalake/ingestion.db`	16,384 bytes (16.0 KB)
Total Ingestion Storage	—	**11.31 MB**

7.1.3 Discussion of Findings

1. Fresh Ingestion Scales Concurrently

Fresh ingestion scales well with the number of clients: **1.68×** from 1 to 2 clients (43.9 ⟶ 73.9 books/s) and **2.55×** with 4 clients (111.9 books/s).

- **Why Ingestion scales where Indexing hits a ceiling:** In section 7.2, persisting the index stops improving after 2 threads, because every thread competes for the single write lock of `datamart.db`. Ingestion has no such shared hot spot: each book is fetched, split and written to its own files (`<book_id>.header.txt` and `<book_id>.body.txt`), so different books do not wait for each other.
- The only shared write is one `INSERT OR REPLACE` per book into `ingestion.db`, a single short statement serialized by SQLite. The near-linear scaling up to 4 clients indicates that it is not a bottleneck at this scale.

2. JIT Warm-Up and Latency Distribution

In Run 1 with 1 client, the maximum latency reached 392.4 ms due to one-time JVM classloading, Javalin/Jetty initialization, and initial JIT tier compilation. By Run 2, the pipeline is fully compiled:

- Median latency (`p50`) drops to **20.35 ms** for fresh ingestion.
- 95th percentile latency (`p95`) stays tightly bounded under **39 ms**.
- Even under 4 concurrent clients generating over 111 operations per second, median latency remains exceptionally low at **27.0 ms**.

3. Saturation of the Idempotent Fast-Path

When books are already available, the service skips fetching, splitting and writing: it looks the book up in `ingestion_log` and publishes the `BookDownloaded` event again.

- With 1 client, throughput rises to **112.0 books/s** (median latency about **7.8 ms**).
- With 2 clients, throughput reaches **168.7 books/s** (median latency about **9.7 ms**).
- With 4 clients, throughput stays at **172.9 books/s**, while the median latency grows to about **15 ms**: the extra clients mostly wait.
- **The ~173 books/s ceiling:** this benchmark does not record where the time of a fast-path request is spent, so the cause of the plateau cannot be identified from these data. Each request still makes an SQLite query and sends a JMS message, and the 4 client threads share one JVM with the service and the broker. Each run handles only 20 requests in about 0.12 s, so these figures are indicative rather than precise.

4. Zero Error Rate and Fault Resilience

Across all 18 benchmark executions (360 ingestion requests), **zero errors were recorded** (`errors = 0`), and every fresh run stored the complete corpus (11.29 MB in 40 files).

5. Resource Footprint

The whole benchmark JVM (service, clients and broker) used on average **7.9 %** of the machine, about 1.9 of its 24 logical processors, with a peak of **28.2 %** (about 6.8 processors). Its Java heap in use peaked at **279.1 MB**. The machine was therefore far from saturated: the limits observed above come from the work of each request, not from a lack of CPU.

6. Limits of the Evidence

- Each run ingests only the 20 corpus books and lasts between 0.12 and 0.84 s, so single runs are sensitive to scheduling noise; the three repetitions show this spread.
- The service, the clients and a non-persistent broker share one JVM, so the figures exclude the persistence of events in KahaDB and do not separate the CPU of the service from that of the clients.
- Books come from a local stand-in, so the real cost of downloading from Project Gutenberg over the Internet is not part of these figures.

7.2 Microbenchmarks of the Indexing components (JMH)

The JMH microbenchmarks measure the indexing code on its own, with no broker, HTTP or other process, so that the cost of each step can be told apart from the cost of coordination measured in section 7.3.

Method

What is measured. Following correction 4 of Stage 1, construction, persistence and incremental update are measured separately, and the complete rebuild on its own:

Benchmark	One operation	Threads
`tokenize`, `parseHeader`, `indexBook`	Tokenizing the body, parsing the header, and storing book 1342 (*Pride and Prejudice*)	1
`analyseCorpus`	**Construction**: reading, parsing and tokenizing the 20 books and counting their terms, in memory	1, 2, 4
`persistCorpus`	**Persistence**: storing the 20 precomputed books in an empty datamart, one transaction per book	1, 2, 4
`buildCorpus`	Both, as the service does it, from the files into an empty datamart	1, 2, 4
`updateBatch`	**Incremental update**: re-indexing an identical batch (the corpus) into a datamart that already holds it	1, 2, 4
`rebuild`	The complete rebuild of `POST /index/rebuild` over a full datamart (sequential, as the service runs it)	1

The threads play the role of the `INDEXING_WORKERS` consumers: each one takes the next book from a shared queue, and the operation ends when the 20 books are done.

Settings (corrections 7 and 8).

- JMH 1.37, one fork per benchmark, with `-Xms2G -Xmx2G`.
- Book 1342: average time, 5 warm-up and 10 measured iterations of 10 s.
- Corpus: single-shot time of one pass, 5 warm-up and 10 measured passes, with an explicit limit of 2 minutes per pass. Logging is reduced to warnings, so that writing a line per book is not measured.
- The GC profiler adds the bytes allocated per operation (`gc.alloc.rate.norm`).
- The datamarts are created in temporary directories on the system disk, with the production SQLite settings (WAL, `synchronous=NORMAL`, `busy_timeout=5000`).
- Every run uses the fixed corpus of `samples/books.txt`.
- All the official figures come from the team's reference machine, with Temurin 21.0.12 and the same commit as the integration benchmarks. The exact command, commit and settings are in `command.txt`, and the machine is described in `host.txt`.

The whole run is one command, described in `indexing-service/README.md`. It writes `benchmarks/results/<day>_indexing-jmh/`: `jmh.csv` (raw JMH results), `jmh.txt` (JMH log), `footprint.csv`, `command.txt` and `host.txt`.

Results

All the figures come from `benchmarks/results/20261105_indexing-jmh/`, measured on the reference machine (13th Gen Intel Core i7-13700KF, 24 logical processors, 31.9 GB of memory, Windows 11, Temurin 21.0.12) at commit `04132ab`, the same commit and machine as section 7.3. 1 MB = 10^6 bytes.

Table 7.2.1. Steps of indexing one book (book 1342), mean ± 99.9 % error over 10 iterations. Source: `.../jmh.csv`.

Step	Time (ms/op)	Allocated (MB/op)
`parseHeader`	0.0063 ± 0.0000	0.004
`tokenize`	8.09 ± 0.06	9.66
`indexBook` (one SQLite transaction)	27.35 ± 1.04	0.69

Table 7.2.2. Time to process the 20-book corpus with 1, 2 and 4 threads, mean ± 99.9 % error over 10 passes. Speed-up is relative to 1 thread. Source: `.../jmh.csv`.

Benchmark	1 thread (ms)	2 threads (ms)	4 threads (ms)	Speed-up 2 / 4
`analyseCorpus` (construction)	224 ± 16	115 ± 18	63 ± 10	1.94 / 3.54
`persistCorpus` (persistence)	1303 ± 405	591 ± 83	630 ± 138	2.21 / 2.07
`buildCorpus` (both)	3020 ± 988	1127 ± 348	788 ± 445	2.68 / 3.83
`updateBatch` (incremental update)	1654 ± 149	1333 ± 154	1199 ± 92	1.24 / 1.38

Table 7.2.3. Complete rebuild against building from scratch and updating the same batch (1 thread). Source: `.../jmh.csv`.

Operation	Time (ms)	Books/s
`buildCorpus`, empty datamart	3020 ± 988	6.6
`updateBatch`, full datamart	1654 ± 149	12.1
`rebuild` (`POST /index/rebuild`)	1679 ± 106	11.9

Table 7.2.4. Memory of indexing the corpus. Source: `.../footprint.csv` and `.../jmh.csv`.

Measure	1 thread	2 threads	4 threads
Java heap before indexing, after a GC (MB)	3.82	3.92	3.93
Peak Java heap while indexing (MB)	208.0	210.5	210.5
Java heap after indexing, after a GC (MB)	3.91	3.93	3.93
Allocated per corpus pass, `buildCorpus` (MB)	254.1	254.1	254.1

The peak heap is the sum of the peaks of the heap pools, so it is an upper bound that depends on the garbage collector. The SQLite page cache is native memory and is not included. The resident memory of the whole Indexing process is in Table 7.3.4.

Discussion

- **Construction scales, persistence does not.** Analysing the corpus in memory is almost perfectly parallel: 1.94× with 2 threads and 3.54× with 4 (Table 7.2.2). Persisting it stops improving after 2 threads (591 ms with 2 threads, 630 ms with 4, within the error). This is the single SQLite writer: extra threads only queue on the write lock. The incremental update, where storing dominates, gains only 1.24× and 1.38×. The integration benchmarks see the same ceiling through the broker: 12.3 books/s with 1 worker, 18.0 with 2, 18.5 with 4 and 18.4 with two replicas (mean of 3 runs, `20261105_indexing-throughput/raw.csv`). A second replica adds no writer capacity, because both processes write the same file.
- **The cost of a book is the transaction, not the text.** For book 1342, `indexBook` takes 27.3 ms against 8.1 ms for tokenizing and 0.006 ms for the header (Table 7.2.1). Tokenizing allocates 14 times more memory than storing, but it runs in parallel; the transaction does not.
- **Rebuild and update cost the same.** The complete rebuild takes 1679 ms against 1654 ms for re-indexing the same batch with 1 thread (Table 7.2.3): reading the ingestion log and checking for removed books is negligible next to the re-indexing itself. With 1 thread both handle about 12 books per second, as fast as the whole pipeline with 1 worker (12.3 books/s), which also re-indexes books already in the datamart. Coordinating through the broker therefore adds little to the cost of indexing a book.
- **Why one thread is penalised.** With 1 thread, `persistCorpus` is more than twice as slow as with 2 (2.21×), which a single writer cannot explain. Each `DatamartStore` call opens and closes its own connection, and when the last connection to a WAL database closes, SQLite checkpoints the whole WAL into the database file and deletes it. With one thread and no other open connection, that happens after every book. With two or more threads the connections overlap, and the costly checkpoint is mostly avoided. With the default of 2 workers the connections of the service usually overlap, so the figures with 2 or more threads are the representative ones; a single worker would pay this checkpoint after every book.
- **An unexplained outlier.** `buildCorpus` with 1 thread (3020 ± 988 ms, individual passes from 2.1 to 3.8 s) is slower than its two parts measured separately (224 ms + 1303 ms), and slower than updating a full datamart. It is the only configuration with that behaviour; with 2 and 4 threads, building is faster than updating (1127 against 1333 ms, and 788 against 1199 ms), as expected because an update must also delete the previous postings. It only affects passes that create a new database file in the system's temporary directory, so we suspect the file system or the antivirus scanning the new files of that machine, but we could not reproduce it elsewhere. We therefore do not draw conclusions from that single figure.
- **Memory.** The index lives on disk. The heap returns to under 4 MB after indexing, whatever the number of threads (Table 7.2.4). While indexing, the peak stays near 210 MB with 1, 2 or 4 threads, and a pass allocates 254 MB, 94 % of it in the analysis (tokenizing and counting). These are short-lived objects, so the garbage collector reclaims them cheaply.

- **Possible improvement.** Keeping one connection per worker open for its whole life, instead of one per book, would avoid the per-book checkpoint with 1 worker and the cost of opening connections. We did not change it after the functional freeze, because that would invalidate the measurements; it is a candidate for Stage 3.

Limits of the evidence

- **One machine, one disk.** The benchmarks measure the code on the reference machine. The cost of the commits depends on its disk and file cache.
- **Small corpus.** With 20 books, the datamart fits in the file cache, so the times reflect CPU and SQLite work rather than disk reads. A larger corpus would make the single writer and the size of the `postings` index weigh more.
- **No coordination.** These figures exclude the broker, HTTP and event publication; section 7.3 measures the whole chain.

7.3 Integration benchmarks

The integration benchmarks measure the whole system as it is deployed on the Stage 2 host: four kinds of process (broker, Ingestion, Indexing and Search), each in its own JVM, coordinating through ActiveMQ and sharing the SQLite files on one disk. They complement the JMH microbenchmarks of section 7.2, which measure single components without coordination.

Method

All the experiments are run by one command, `java -jar benchmarks/integration/target/integration-bench.jar`, on the reference machine. Its procedure is in `benchmarks/README.md`.

Host.

Item	Value
CPU	13th Gen Intel Core i7-13700KF, 24 logical processors
RAM	31.9 GB
Disk	1 TB NVMe M.2 SSD (Kioxia Exceria G2, PCIe 3.0 x4)
OS	Windows 11 (amd64)
JDK	Eclipse Temurin 21.0.12 (OpenJDK 64-Bit Server VM)
Commit	`04132ab`
Date	2026-11-05, started 17:54 UTC, after the ingestion and JMH runs had finished (17:53 UTC)

The same host, JDK and commit are used for every table of this section and for the JMH results.

Controlled conditions.

- **Fresh system per run.** Every run starts with a new `DATA_DIR` and starts the services from their jars, each with `-Xmx1g`.
- **Fixed corpus.** It loads the 20 books of `samples/books.txt` through `POST /ingest`. The books are served from `samples/books` by a local stand-in for Project Gutenberg, so network latency is never part of a measurement and the corpus cannot change between runs.
- **Same settings as production.** SQLite runs with `journal_mode=WAL` and `synchronous=NORMAL`. The broker persists messages in KahaDB.
- **Load generator on the same host.** It competes for the same CPU, which is part of the single-host deployment being measured.

Factors. Following section 6.2 of the assignment, one factor changes at a time, keeping the corpus, the workload and the host fixed:

Factor	Values	Meaning
Client concurrency	1, 2, 4, 8, 16, 32, 64	Concurrent search clients, each with one connection and no think time (closed loop)
Search replicas	1, 2	Search processes over the same datamart; clients spread requests round-robin
Indexing worker threads	1, 2, 4	`INDEXING_WORKERS`: consumer threads of one Indexing process
Indexing replicas	1, 2 (with 2 threads each)	Indexing processes competing for the same queue

Client concurrency is reported separately from internal threads and replicas.

What is recorded.

- **Search latency.** Per concurrency level: a 10 s warm-up that is not recorded, then 30 s of measurement. Every request is stored with its latency and the number of matches it returned (`raw.csv`). The percentiles use the nearest-rank method over successful requests.
- **Indexing throughput.** 200 `BookDownloaded` events (10 rounds of the corpus) are sent straight to the queue and timed until the 200 `BookIndexed` events have arrived. Each configuration is run 3 times.
- **Coordination cost.** On an empty system, each book is ingested alone and timed until its `POST /ingest` response, its `BookIndexed` event and its visibility in `GET /books/{id}`. Then the same re-indexing is timed through the broker and as a direct `POST /index/update`, with no broker in the path.
- **Resources.** Throughout, the CPU % (100 % = one core) and resident memory of every process are sampled every second (every half second in the shorter indexing and coordination runs), and the disk used by each store is recorded.

Correctness of every configuration. Throughput only counts if the results stay correct:

- **Search.** Every request of the same word must return the same number of matches, whatever the concurrency or the replica (the `total` column of `raw.csv`). The exact `book_ids` are checked by the API checks of section 4.4.
- **Indexing.** After each throughput run, the index must still hold exactly 20 books and `darcy` must match exactly one book (`index_ok`), so re-indexing never duplicates anything. The Indexing test suite also checks, row by row, that 1, 2 and 4 threads and 2 replicas build the same index.

Results

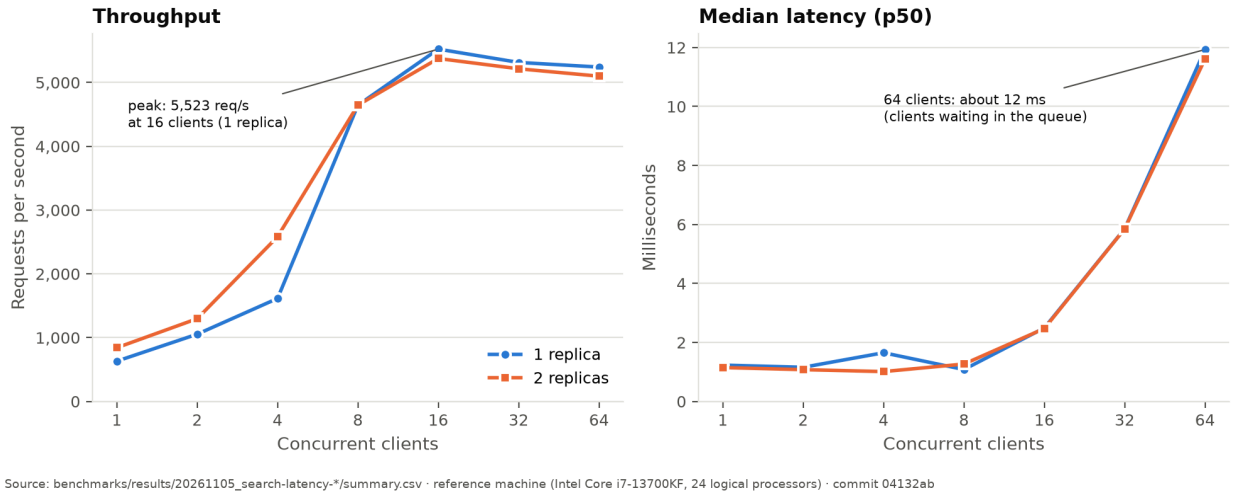
No request failed in any experiment, and every configuration kept correct results:

- **Search.** Over the 1.47 million measured requests, each word always returned the same number of matches, with 1 and with 2 replicas (`darcy` 1, `wonderland` 1, `monster` 14, `fortune` 18, `heart` 19, `truth` 19, `death` 20, `love` 20, `nothing` 20, `time` 20).
- **Indexing.** All 12 throughput runs ended with `index_ok = true`.

Search latency and throughput

Table 7.3.1. `GET /search` with 1 and 2 Search replicas; 30 s of measurement per level, after 10 s of warm-up. Sources: `benchmarks/results/20261105_search-latency-1-replica/summary.csv` and `benchmarks/results/20261105_search-latency-2-replicas/summary.csv`.

Clients	1 replica: req/s	p50 (ms)	p95 (ms)	p99 (ms)	2 replicas: req/s	p50 (ms)	p95 (ms)	p99 (ms)
1	631	1.23	2.80	4.91	842	1.15	1.45	1.77
2	1,054	1.16	4.70	15.54	1,297	1.08	1.78	15.29
4	1,615	1.65	13.04	16.07	2,582	1.01	1.57	15.81
8	4,645	1.08	2.13	16.00	4,646	1.27	2.06	16.01
16	**5,523**	2.49	4.13	16.48	**5,376**	2.47	4.38	16.45
32	5,315	5.86	6.95	13.70	5,215	5.85	7.52	18.79
64	5,242	11.93	13.44	22.09	5,099	11.61	15.56	41.25



Source: benchmarks/results/20261105_search-latency-*/summary.csv · reference machine (Intel Core i7-13700KF, 24 logical processors) · commit 04132ab

Figure 7.3.1. Throughput (left) and median latency (right) of `GET /search` against the number of concurrent clients, with 1 and 2 Search replicas. Data of Table 7.3.1; drawn by `docs/report/figures/plot_search_latency.py`.

- **Saturation at 16 clients.** Throughput grows almost linearly up to 8 clients and peaks at 16 clients, with about 5,500 requests per second. Beyond that point it stays flat or drops slightly, and the latency grows in proportion to the number of clients. With 64 clients the mean latency is 12.2 ms, which is 64 clients divided by 5,242 requests per second (Little's law). The extra clients only wait in a queue. Below saturation a search takes about 1–2.5 ms at the median.
- **A second replica does not raise the ceiling.** With 2 replicas the peak is the same within a few percent (5,376 against 5,523 requests/s), and so is the latency at 16 clients or more.
- **The limit is not inside the Search process.** At peak load the Search processes together used at most 876 % CPU with one replica and 887 % with two, that is, about 9 of the 24 logical processors (Table 7.3.4). The two replicas split the same total work (485 % and 457 % at most) instead of adding capacity. If one Search process were the bottleneck, a second one would have raised the ceiling. The constraint is therefore shared by the whole host. The most likely cause is the load generator, which runs on the same machine with up to 64 client threads; its CPU was not sampled, so this cannot be confirmed from the data. The Stage 2 deployment cannot separate the two; on the cluster, clients and servers will run on different machines.
- **Low concurrency.** With 1 to 4 clients, 2 replicas gave higher throughput and much lower p95: 4 clients reached 2,582 requests/s against 1,615, with a p95 of 1.57 ms against 13.04 ms. With one client the requests are sequential, so the gain cannot come from parallelism. These runs do not show the cause, so we do not attribute this difference to replication.
- **A latency floor of about 16 ms in the tail.** From 2 to 16 clients, p99 stays between 15.3 and 16.5 ms in both configurations, although p50 is about 1 ms. This value matches the default 15.6 ms timer interval of Windows, which suggests that the tail is set by the operating system scheduler rather than by the search itself. We did not verify this with another timer setting.

Indexing throughput, threads and replicas

Table 7.3.2. Books indexed per second from the queue: 200 `BookDownloaded` events per run, mean of 3 runs with min–max, and the CPU used by the Indexing processes (mean of the per-run averages). Source: `benchmarks/results/20261105_indexing-throughput/raw.csv` and `resources-*.csv` of the same directory.

Indexing processes × threads	Books/s (min–max)	Speed-up vs 1 × 1	Indexing CPU %	`index_ok`
1 × 1	12.3 (12.1–12.5)	1.00	92	3/3
1 × 2	18.0 (17.8–18.1)	1.46	134	3/3
1 × 4	18.5 (18.1–18.7)	1.50	142	3/3
2 × 2	18.4 (18.3–18.5)	1.49	140	3/3

- **A ceiling of about 18.5 books per second.** A second thread improves throughput by 46 %, but 4 threads or 2 processes add almost nothing more. The three repetitions of each configuration are within ±2 %, so the plateau is not noise.

- **The extra threads wait.** With 1 thread the process uses about one core (92 %). With 2 threads it uses 134 %, and with 4 threads, or with 2 processes, it still uses only about 140 %. Doubling the threads did not double the CPU used: the additional threads spend their time waiting, not working.
- **They wait for the SQLite writer.** SQLite lets one transaction write a database at a time, and every book is one transaction (section 5.2). Reading and tokenizing a book run in parallel, which explains the gain of the second thread. Committing is serialised, so beyond two threads the workers queue on the write lock. The JMH results of section 7.2 show the same split without the broker: analysing the corpus scales 3.5× with 4 threads, whereas persisting it stops improving after 2.
- **Replicas do not help either.** A second process writes the same `datamart.db`, so it competes for the same lock. 2 × 2 performs like 1 × 4.

Coordination overhead

Table 7.3.3. Per-book times in milliseconds over 3 runs × 20 books, each book processed alone. Source: `benchmarks/results/20261105_coordination/summary.csv`, `chain.csv` and `direct-vs-event.csv`.

Measure	Median	p95	Max
`POST /ingest` response (local stand-in download, split, atomic write, log, publish)	19.4	38.6	304.3
Until its `BookIndexed` arrives	59.7	104.8	356.7
Until it is searchable (`GET /books/{id}` = 200)	64.3	109.2	429.4
Re-index with direct `POST /index/update` (no broker)	72.9	117.7	160.2
Re-index through the broker (`BookDownloaded` → `BookIndexed`)	73.9	119.6	172.7

- **The whole pipeline takes about 64 ms per book.** A new book becomes searchable at the median 64 ms after it is requested: 19 ms in Ingestion, about 40 ms more until Indexing has committed it and announced it, and about 5 ms more until the client sees it in Search.
- **The maximum is the first book of each run.** It is the first request served by freshly started JVMs: book 11, searchable after 429, 331 and 329 ms in the three runs. The p95 values describe the warm system.
- **The broker costs about 4 ms per book.** Re-indexing the same book took a median 72.9 ms directly and 73.9 ms through the broker. Compared book by book, the broker path was slower by a median of 3.8 ms (mean 3.3 ms over the 60 pairs). That is about 5 % of the cost of indexing a book, and it pays for the persistent send, the delivery to a consumer, the JMS transaction and the `BookIndexed` event. In return the services are decoupled and a crash loses no book.
- **Re-indexing costs more than indexing.** Indexing a new book (in the chain) is faster than re-indexing it (in the last two rows), because re-indexing must also delete the previous postings of the book. Section 7.2 measures the same effect.

Resource usage

Table 7.3.4. CPU and resident memory per process during the search load (all levels from 1 to 64 clients), with every service started with `-Xmx1g`. 100 % CPU is one busy logical processor out of 24. Sources: `resources-summary.csv` and `resources.csv` of `benchmarks/results/20261105_search-latency-1-replica/` and `..._search-latency-2-replicas/`.

Process	1 replica: mean CPU %	Max CPU %	Max RSS (MB)	2 replicas: mean CPU %	Max CPU %	Max RSS (MB)
search-1	424	876	447	220	485	446
search-2	—	—	—	218	457	443
indexing-1	0.1	2.8	328	0.1	2.8	332
ingestion	0.0	1.4	294	0.1	2.8	293
broker	0.2	5.7	103	0.1	4.3	105
All services (sum per sample)	425	876	—	439	887	—

While books are being ingested and indexed (coordination experiment, last run), Indexing averaged 76 % CPU with a peak of 609 MB of resident memory, and Ingestion averaged 17 % with a peak of 293 MB (`benchmarks/results/20261105_coordination/resources-summary.csv`).

- **Memory is mostly the JVM.** During the search load each process uses 100–450 MB, well below its 1 GB heap limit. The index lives on disk and is read through SQLite and the operating system cache, so Search memory does not grow with the load.
- **Under search load only Search works.** The other services stay idle, and the busiest moments of the whole system use about 9 of the 24 logical processors.

Table 7.3.5. Disk used by the 20-book corpus (1 MB = 10^6 bytes). Source: `benchmarks/results/20261105_search-latency-1-replica/disk.csv`.

Store	Size (MB)
Datalake (headers and bodies)	11.29
Ingestion log (`ingestion.db`)	0.02
Datamart (`datamart.db` with its WAL)	7.23
Broker store (KahaDB)	67.31

The datamart takes 64 % of the size of the text it indexes. The broker store is the largest, but only because KahaDB preallocates its journal files: the messages themselves are a few hundred bytes each.

Limits of the evidence

- **One host.** Every number is measured on a single host, where the services, the broker and the load generator share the CPU, the memory and one disk. A speed-up measured here shows how the design uses one machine, not how it will scale on the Stage 3 cluster. In particular, the search ceiling could not be attributed to a single component, because the load generator could not run on another machine.
- **Host CPU.** The CPU of the host is reported as the sum of the service processes. The load generator and the rest of the operating system were not sampled.
- **Small corpus.** The 20-book corpus keeps the experiments reproducible but small. The datamart fits in the operating system's file cache, so the search latencies reflect CPU and SQLite work, not disk reads.
- **Untested configurations.**
  - Search replicas beyond 2 were not tested; on one host they can only share the same cores.
  - Ingestion replicas were not tested: ingestion is bound by Project Gutenberg, which is replaced here by a local stand-in.
  - Concurrent ingestion of many books was not measured end to end; the ingestion benchmark of section 7.1 covers it.

8. Conclusions and future improvements

Stage 2 runs every process on one host. The architecture was chosen so that most of it carries over to a cluster: services that talk only through HTTP and broker events, idempotent indexing, acknowledgement after commit, and every data file owned by exactly one writer. Several assumptions, however, only hold because everything shares one machine. This section lists them and how each one will change.

Assumptions that depend on a single host

Assumption in Stage 2	Where	Why it breaks on a cluster
Every address is `localhost` with a fixed port (`BROKER_URL`, 7001–7003, replica ports)	Configuration, scripts, load client	Processes run on different machines
The datalake is a local directory, and events carry **absolute local paths** (`header_path`, `body_path`)	Ingestion → `BookDownloaded` → Indexing	An Indexing node cannot open a path on another node's disk
One `datamart.db` file is written by Indexing and read by every Search replica	Indexing, Search	SQLite is an embedded database without a server; it needs the file on the same machine
WAL readers see a consistent snapshot through shared memory (the `-shm` file)	SQLite readers and writer	WAL needs every process on the same host; it does not work over a network file system
Indexing opens `ingestion.db` read-only from the datalake disk	Indexing administrative endpoints	Same as above
SQLite serialises writers with a file lock, so concurrent Indexing replicas are safe	Indexing replicas	The lock does not coordinate writers on different machines
"Only one rebuild at a time" is an in-process flag	`IndexingService`	It does not stop a rebuild running on another node
One embedded broker with a local KahaDB store	Broker	A single point of failure; its store is lost if that machine is lost
Search replicas are reached by round-robin clients that know every port	Load client	Real clients need one entry point

How the architecture will evolve

Communication. The services already exchange only HTTP requests and versioned JSON events, so their code does not change. The broker URL becomes a list of addresses with ActiveMQ's `failover:` transport, which every client already uses, so a client moves to another broker when one fails. The broker itself becomes a replicated pair or network of brokers with replicated storage, so accepted events survive the loss of a node.

Storage access.

- **Datalake.** It moves from local paths to shared storage reachable from every node, such as a distributed file system or an object store. `BookDownloaded` would then carry a storage key instead of an absolute path. This is the one contract change, and the event `version` field exists for it.
- **Datamart.** A single SQLite file cannot be shared, so the inverted index has to be distributed:
  - a server database shared by all nodes, or
  - an index partitioned by term or by book over several nodes, each partition with one owner that writes it and replicas that serve reads.

  Partitioning by term keeps a one-word query on one partition but spreads multi-word queries. Partitioning by book makes indexing local but sends each query to every partition and merges the partial top-k results. Since queries greatly outnumber book additions, the trade-off will be decided by measuring both on the cluster.

Distribution of work.

- **Indexing.** The queue already distributes the books: competing consumers on any node take one message at a time (prefetch 1), so adding Indexing nodes needs no change. With a partitioned index, each book's postings are routed to their partitions.

- **Search.** Search replicas are stateless apart from their cache, so they go behind a load balancer or an API gateway, which also becomes the single address for the front-end and the place to configure CORS.

Consistency. Indexing is at-least-once and idempotent, and a book becomes visible in one commit, so duplicates and retries cannot corrupt the index. On a cluster the same guarantees must hold per partition: a book's postings in several partitions have to become visible together, or the API must state that a newly indexed book may briefly be found by some words before others. Search caches stay correct because they are emptied on every `BookIndexed` and disabled while their node is cut off from the broker. This rule already holds on one host.

Recovery from unavailable nodes.

- **What already carries over.**
  - A failed Indexing node leaves its unacknowledged message to be redelivered to another consumer, as the crash test of the Indexing suite shows on one host.
  - Ingestion answers `503` when it cannot publish, and the client retries.
  - Search keeps answering while the broker is unavailable.
- **What is new.** Timeouts between nodes have to be explicit and short, so that a slow node is treated as unavailable. Health checks have to move from "the process answers" to "the process can reach its storage and broker", so the load balancer stops routing to a node that is up but cut off.
- **Locks.** Any coordination that relies on in-process locks (the rebuild flag) or on file locks (the SQLite writer) has to become a distributed lock or a single elected owner per partition.

What the Stage 2 measurements tell us

The integration benchmarks were designed to test three hypotheses about where the single host limits the design. The official figures of section 7 settle them as follows.

- **Indexing is bounded by the single SQLite writer: confirmed.**
  - Throughput rises from 12.3 to 18.0 books per second with a second thread, then stays at 18.5 with 4 threads and 18.4 with two processes (Table 7.3.2).
  - The extra threads do not use more CPU: about 140 % against 134 % with 2 threads. They wait for the write lock.
  - The JMH results show the same split without the broker: analysing books scales with threads, persisting them does not (section 7.2).
  - On the cluster, more writers on the same file cannot raise this limit, but an index partitioned over several machines, each partition with its own writer, can. This is the main reason to distribute the datamart.
- **Search is bounded by the host, not by one Search process: partly confirmed.**
  - Throughput peaks at about 5,500 requests per second with 16 clients, and a second replica does not raise that peak (Table 7.3.1).
  - The Search processes then used about 9 of the 24 logical processors, and two replicas only split that work between them.
  - The limit is therefore shared by the whole machine, where the load generator also runs. It could not be attributed to one component with the evidence of a single host.
  - On the cluster, clients and replicas on different machines will show whether Search replicas scale out, which is what the stateless design of the service is meant for.
- **Coordination through the broker costs little: confirmed.**
  - Re-indexing a book through the broker took a median of 3.8 ms more than doing it directly, about 5 % of the cost of indexing the book (Table 7.3.3).
  - A new book is searchable about 64 ms after it is requested.
  - The broker is therefore not the bottleneck of the pipeline, and its cost buys what a cluster needs most: decoupled services, redelivery after a crash and no lost book.

Strengths.

- Correct results under every tested configuration: no failed request among 1.47 million, the same answers from both replicas, and identical indexes after duplicates, crashes, more threads or more processes.
- Low latency below saturation: about 1–2.5 ms at the median.
- A choreography whose overhead is small.

Weaknesses.

- The single SQLite writer caps indexing at about 18.5 books per second on this host.
- Each `DatamartStore` call opens a new SQLite connection, which adds a WAL checkpoint per book with a single worker (section 7.2). Keeping one connection per worker is the first optimisation to make.
- The search ceiling could not be traced to a single component on one machine.